QBRAIN.AI · DESIGN PACKET

Federation

Independent nodes.
Shared, versioned project history.

Architecture, BDPv4 implementation plan,
three-system workflows and requirements

Planning baseline only · revised 10 November 2026

Substantive revision awaiting independent review

Ph0 evidence gates remain before implementation.
Federation is disabled. DEPLOY HOLD remains active.

Compiled by Codex from the current planning sources. Includes the architecture, phase plan, all 15 workflows, full current contracts and acceptance criteria, and applicable gap-check requirements. Historical source copies and duplicated specification exports are excluded.

DOCUMENT GUIDE

Contents

1. Architecture
2. BDPv4 implementation plan
3. Workflows and acceptance inventory
4. Illustrated examples: convergence and conflict review
5. Current contracts and full acceptance criteria
6. Current-scope coverage

The architecture and plan describe proposed behavior. Test names and requirement criteria are future acceptance obligations, not passing implementation results. Current review status and feasibility gates are stated in the plan.

Federation architecture: independent nodes with Dolt history

Author: Codex. Date: 2026-11-10. Status: proposed architecture for the requested new plan; no implementation or deployment performed. Supersedes the forwarding-first federation completion draft for the new versioned federation mode. Existing DirectProxy behavior remains a separate supported feature.

1. Authority and scope

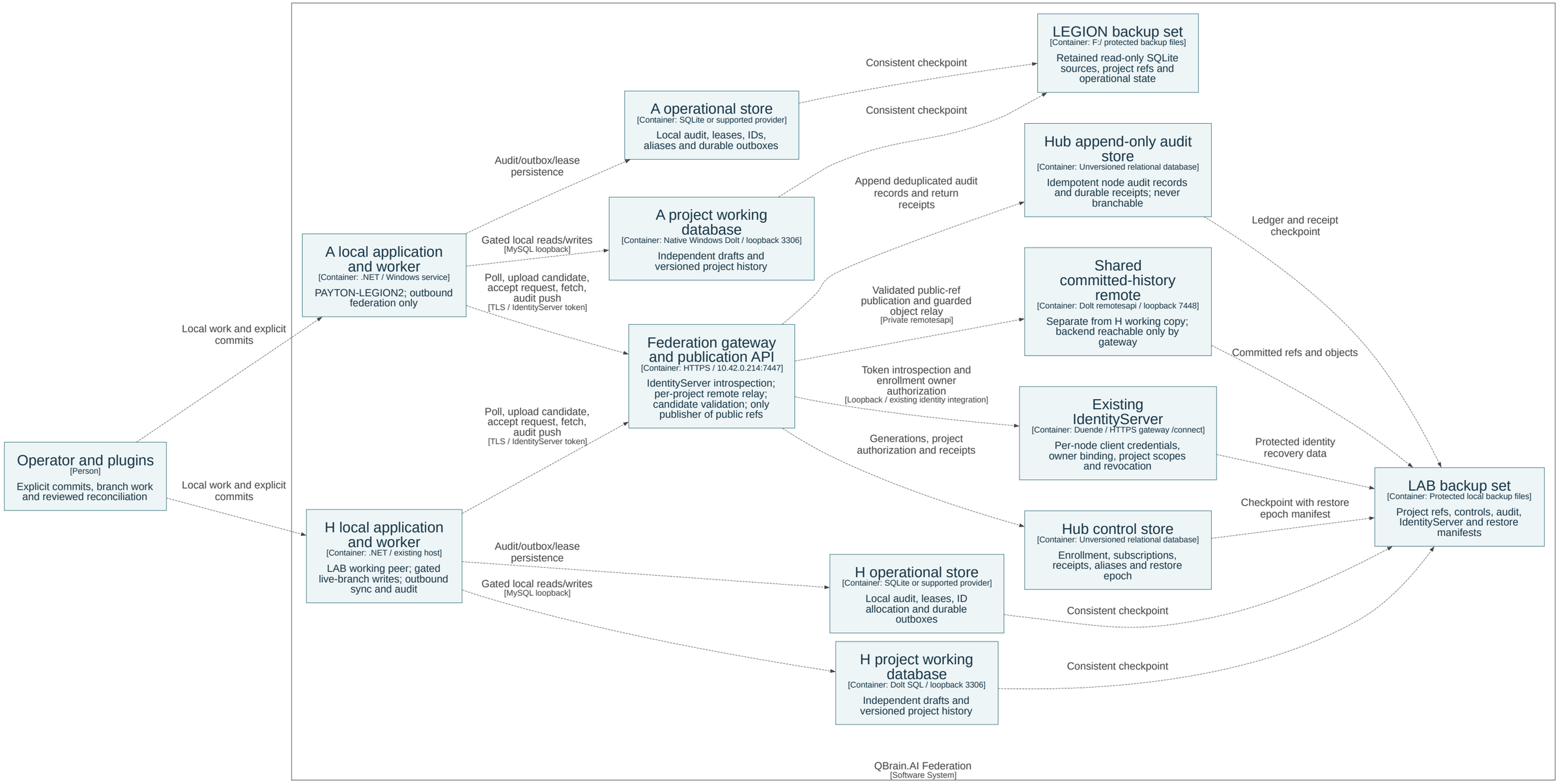
The operator requires LAB-OMARCHY as hub, independent local work on every system, two-way synchronization, two-character creator tags in new IDs, branches in the initial release, and mandatory review of merge conflicts. Pull requests are out of scope. Local **version-history commits**, never ordinary edits or SQL saves, trigger synchronization. Nodes poll a durable hub control table periodically or on demand. A hub request must never implicitly commit a user's working changes.

The operator additionally requires Dolt as the **fourth supported database**, retaining SQLite, PostgreSQL and SQL Server, with appropriate schema migrations and a separate data migration tool from all three original providers. This document adopts that planning direction. The exact maintained EF Core provider/package version and pinned Dolt image must pass the compatibility gate before implementation dependencies are installed. Dolt's relational cell/schema merge is the proposed interpretation of Git-style merging; literal line-level text merging is not claimed.

The three-system example uses H = LAB-OMARCHY, A = PAYTON-LEGION2, and B = NODE-B (illustrative second node, not a discovered host). All three run the same application services and local commit/sync worker. The hub additionally hosts the shared remote and control table. This is eventual consistency, not a distributed transaction across all three hosts.

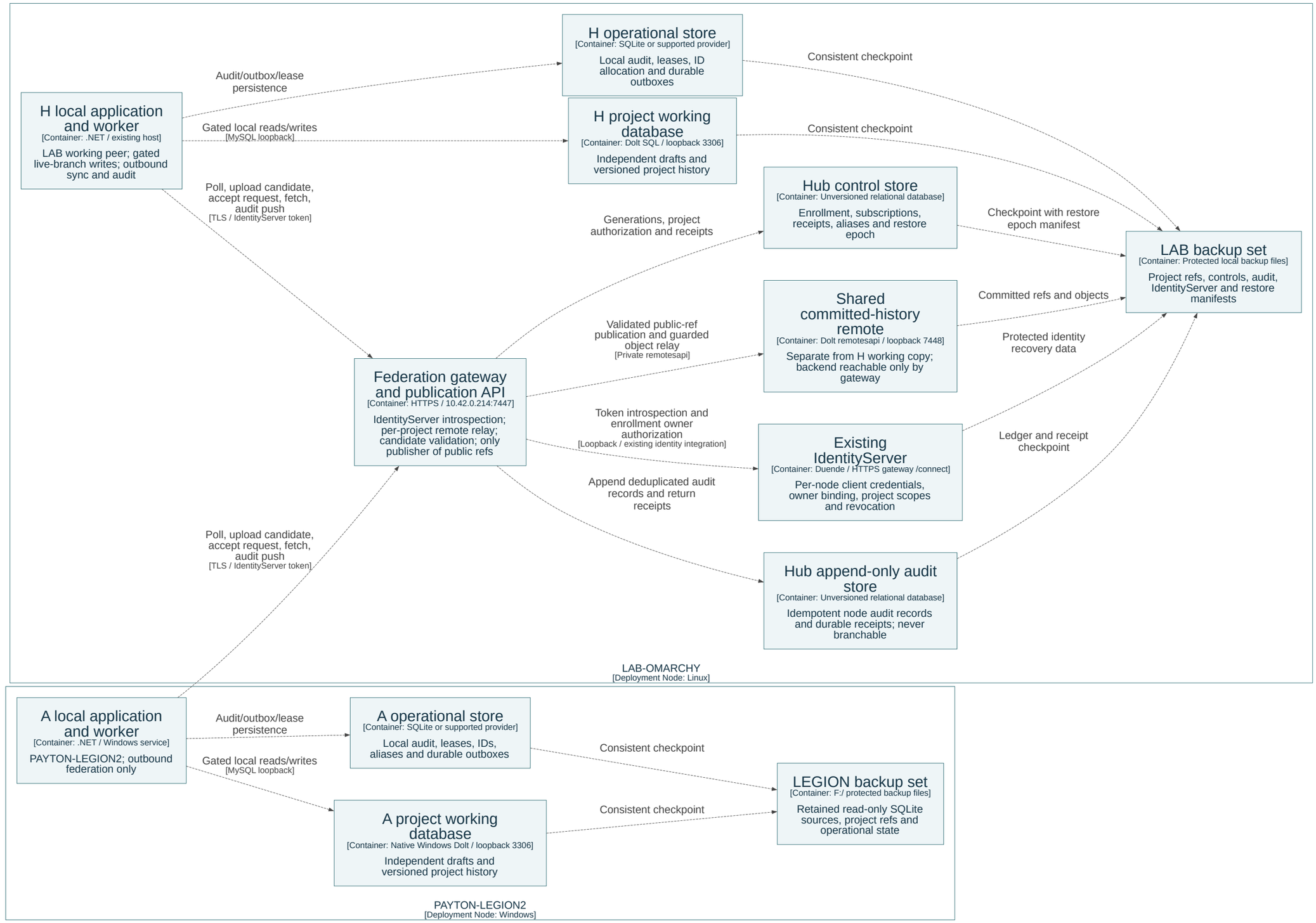
2. Topology and storage boundaries

The authoritative model is Structurizr workspace, with generated container and deployment views. The deployment view uses only LAB-OMARCHY and PAYTON-LEGION2. NODE-B in U01-U12 is a logical test peer, not a claim of a third physical host.



Container View: QBrain.AI Federation

Federation container view



Deployment View: QBrain.AI Federation - Lab target

Federation deployment view

Every host uses a local Dolt SQL-server process/container; .NET reaches it through a MySQL-protocol EF provider and its driver selected in Ph0, subject to compatibility and license review. Dolt is a separate database engine, not an extension installed in an existing MySQL database. It does not wrap arbitrary SQL Server/PostgreSQL/SQLite storage.

For versioned projects, use one Dolt repository/database per logical project on each host. This makes clone/fetch boundaries match project authorization and prevents a project clone from carrying unrelated projects. Local directory paths map to an immutable ProjectId; paths are not distributed identity. The provider resolver must support project-specific routing for enrolled projects; default provider behavior remains unchanged for other workspaces.

The hub's shared remote is a **separate repository/database from the hub's working copy**, optionally served by the same Dolt process. No user edits the shared remote's working set. This matters because a push to a live SQL-server branch can fail when that branch has uncommitted changes. The hub's own draft work must not block A and B from exchanging commits.

Keep host configuration, credentials, enrollment, control requests, receipts, local audit tables and ID-allocation state in a separate unversioned control store. It may use any supported provider, including a separate Dolt database that is never published. Do not merely hide these rows with filters or `dolt_ignore` inside a repository that will be cloned. FR-MCP-AUDIT-001 and TR-MCP-AUDIT-004 are amended: preserve permanent local audit AND deliver it append-only to the hub unversioned audit store. Exclusion from Dolt project history does not mean exclusion from federation. Audit delivery is independent of version commits; section 11 defines it.

Versioned project data includes TODOs, requirements and mappings, use cases, workspace-scoped memories, portable project configuration and explicitly promoted sanitized session-log snapshots. Continuous session-log capture writes local operational storage, without an editing lease; promoting a snapshot is an explicit versioned edit visible in commit preview. Automatic writers must not modify versioned working sets. Ph0 classifies every entity, relationship, writer and auto-updated column as project-versioned, local-operational, derived or explicitly excluded; no unclassified writer/table is permitted. Global memories, machine paths, tokens and host-specific agent/runtime configuration remain local. Derived search indexes, caches and projections live outside versioned databases and rebuild after accepted head changes. Source-code Git repositories and filesystem assets retain their existing Git/file workflows.

Context topology is explicit: original providers and standalone Dolt keep the full `McpDbContext` and existing/baseline migrations. Versioned mode uses a Dolt-only project context plus a control context with additive migrations for each of the four providers. Ph0's relationship manifest names every cross-database reference, replacing impossible database FKs/joins with stable soft references and application validators. Historical migration files are never edited; new additive migrations are allowed where the manifest requires.

3. Local edits, commits and branch ownership

Ordinary business reads/writes always execute locally. Saving creates durable local working changes but neither publishes them nor creates a version commit. The server exposes status, diff and commit preview. An explicit commit includes ProjectId, branch, expected head, expected working-set fingerprint, author, message and request id. The preview identifies exactly which working changes will be committed. A changed fingerprint rejects the request for a new preview.

One editing owner per project/branch working set is the initial policy. A durable branch lease, with fencing token checked by every mutating service, prevents one agent's commit from accidentally including another agent's unfinished work. Other sessions may read or use a different branch. Expired ownership with dirty data requires explicit takeover review; no automatic discard or commit. This policy applies to the hub too. A future finer-grained staging UX is not needed for this release.

Connections and DbContexts are bound to ProjectId and branch for their entire lifetime; never change a shared connection's branch underneath an active request. Pools must not leak branch/session state. APIs, search, caches and plugin context include branch identity. A branch switch does not switch the source-code Git branch unless separately requested.

Exactly one host-local server process owns live-branch writes to each project database, enforced by an exclusive process/storage ownership lock and server-only credentials. Its per-(project,branch) write gate serializes lease issuance/takeover, fence checks, edits, preview-hash validation and Dolt commit. Expiry denies new commands; an already admitted transaction holds the gate through completion, and takeover waits. Process death rolls back uncommitted SQL work before a replacement writer opens. There is no claimed atomic transaction between control storage and Dolt. Use a Dolt-provided working-set hash, verified in Ph0, not a custom approximate hash. Put the unique commit requestId in a commit trailer; recover lost replies by finding that ID in retained Dolt history and returning the same hash. Never issue a second commit merely because the operational receipt is missing.

After a successful version commit, the local worker records publication intent. Because a version commit and an operational ledger write may not share a transaction, startup/reconnect scans compare local branch history with durable publication receipts and reconstruct missing intents. Repeated scans must not create extra commits. Existing imported commits do not count as newly authored commits for echo scheduling.

Sync-worker mutations of a live branch, including its dirty/lease check, merge into live and head move, run under that branch's write gate holding a fenced lease. Temporary integration may run against a captured snapshot outside the gate, but final application reacquires the gate and rechecks head, working-set fingerprint and editing ownership. If B5 isolates the worker into another process, it holds remote credentials and integration-branch rights only; it requests live-branch moves through the owning server's gate API. Every live-branch write remains covered by the exclusive writer lock.

New record IDs contain the assigned two-character creator tag plus a durable nonreused sequence. Proposed tag alphabet: uppercase ASCII letters/digits. Actual host tag assignments require registration before first tagged creation. The allocator is outside branches so branching or reverting cannot reuse an ID. Restored/cloned writers must prove a safe high-water mark or enroll with a fresh never-reused tag. Replication preserves IDs. Legacy collision resolution preserves permanent origin-scoped aliases and updates relational identity links through the reviewed mapping. It never rewrites historical commit, PR, session-log, memory or document text. Equal old IDs are not proof of shared identity; different IDs are not proof of different work. Section 12 covers duplicate detection and human ID reservations.

Ph0 must lock the exact grammar per entity family, distinguish system-generated and human-assigned IDs, and enumerate validators, traceability tooling, plugin regexes, sort and display rules. Existing canonical requirements and legacy IDs remain valid. No blanket renumbering of the 48 newly registered planning records is intended.

4. Publication and synchronization protocol

Use Dolt for object transfer, ancestry, diff, branch creation and merge. Do not implement a competing event-replay database, merge engine or reliable messaging broker.

The shared remote keeps named public branches and private upload refs. A node may upload only its authenticated project-scoped candidate namespace in a physically isolated upload repository. The gateway controls the backend; no node receives credentials capable of writing the public repository. The hub publication API owns advancement of public branch heads; ordinary node/plugin credentials must not bypass it. Before accepting a candidate it verifies project membership, ancestry, schema compatibility, constraints and domain invariants. The candidate must be a descendant of the expected published head. Use Dolt fast-forward protection plus one durable fenced publisher per project/branch; never force-push a public branch. A stale expected head returns `head_changed`; the node fetches and reconciles again.

This candidate-upload/accept step is **not a PR workflow**: no PR entity, reviewer assignment, approval voting or forge. It is a guarded publication boundary. Ph0 must demonstrate that the chosen remote transport can enforce these permissions, including direct remote calls; SQL branch permissions alone are not assumed to protect remotesapi. If the transport cannot provide this boundary, select a supported isolated transport arrangement and review it before implementation, rather than exposing writable public refs.

Normal worker sequence:

1. Authenticate, read current published heads, and retrieve its pending control requests. Also inspect unpublished local commits, even if no control request exists.
2. Fetch committed history into remote-tracking refs. Fetching does not commit or overwrite local drafts.
3. If the target local branch is dirty or has an active edit lease, record `waiting_local_work`. Keep the prior readable local state. The user may finish and commit, explicitly discard with normal safeguards, or work on another branch; sync chooses none of these for them.
4. On a clean branch, use a temporary integration branch based on the current local head. Fast-forward if possible; otherwise use Dolt's merge. Preserve the old live branch until the candidate passes review/validation.
5. If Dolt reports conflicts or validation detects a domain violation, persist `needs_review` with base/local/incoming heads and affected entities. Nothing silently wins. Independent projects and branches continue.
6. For a valid clean merge, move the local live branch using expected-head/fencing checks. Publish any new local/merge commits through candidate upload and the hub acceptance API. If the hub advanced, repeat from fetch without creating duplicate effects or force-pushing.
7. Persist receipt with resulting local/published heads, refresh branch-scoped derived indexes and report the outcome. A receipt proves this synchronization target was reached, not that every other node is already current.

Both directions are node-initiated. H's local worker uses the same authenticated gateway endpoint, never a file:// URL into a repository served by a running SQL server. No inbound connection to A or B is required. Successful publication advances each subscriber's desired state. Periodic remote-head reconciliation repairs a missed control-store update. Correctness does not depend on an atomic transaction across Dolt and the control database.

5. Hub control table

Implement the logical queue as **one durable state row per subscription**, not a command-message broker. Initial polling default: every 30 seconds with up to 20 percent jitter; also poll immediately on reconnect and explicit sync. Local version commits wake the worker immediately. Backoff transient network failure exponentially from 1 second to a 60-second cap, with cancellation; Ph0 locks final settings against measured behavior.

`SyncSubscription`: unique key `(NodeId, ProjectId, Branch)`; HubEpoch, PublishedHead, PublishedGeneration, DesiredGeneration, SyncRequestedUtc, NodeAcceptedHead, NodeReportedGeneration, Outcome, LastSeenUtc, WorkerEpoch and ErrorCode. Publication advances PublishedGeneration and DesiredGeneration. Explicit on-demand requests advance DesiredGeneration even when the published head has not changed; timestamps are informational, never ordering authority. Poll returns authorized rows with an unsatisfied desired generation or a visible blocked outcome. Locally unpublished commits independently wake the worker even if every row is satisfied.

Generation ordering is scoped to a non-reused HubEpoch. Restore installs a fresh epoch before any endpoint reopens and forces ancestry reconciliation; a node never discards a head because a restored counter is lower. The authenticated worker claims a monotonic WorkerEpoch within the current HubEpoch on reconnect/restart; reports compare epoch and the generation actually observed. A late old-epoch report is rejected. A report for generation g can record its outcome without clearing desired generation g+1. A completed report requires NodeAcceptedHead to include the target and records a durable operation receipt; retries of the same report return that receipt. A transient failure remains unsatisfied with node-local retry scheduling. `waiting_local_work`, `needs_review`, `incompatible_schema` and `denied` are visible non-success outcomes, not completed requests. Do not add per-message leases, acknowledgement deletion or coalescing machinery to this state table.

The hub verifies claimed ancestry against remote objects and the recorded publication receipt for the observed target generation. A node-attested accepted head absent from the remote is provisional until its ancestry is uploaded or published; it cannot complete the subscription generation on assertion alone. Local status may still report that local integration succeeded.

The hub owns a row for each subscribed node, including its own local workspace worker. A committed change is not considered delivered to a node until its recorded accepted head includes the target commit. This permits a later merge commit to satisfy an earlier target without pretending head strings must be equal.

6. Conflict review and branches

A review item records project, branch, stable conflict id, common ancestor(s), local head, incoming head, candidate branch, conflict cells/schema, validation failures, author identity, decision and resulting merge commit. The server exposes structured diff/history and resolution preview. A human or explicitly authorized agent selects a resolution; apply requires unchanged source heads and reruns constraints/domain validation. A stale review is refreshed, not applied blindly. Retain `review/<conflictId>` and candidate refs locally and on the remote until a durable resolution receipt permits release under Ph0's retention policy; control-table references alone are not GC roots.

Sharing `feature/search` publishes that branch; it does not merge it into `main`. A and B can independently check out the same feature history. Merging feature into main is a separate explicit command with expected heads, normal review rules and a new version commit. Protected-branch rights govern who may perform it. Branch deletion/rename requires explicit authorization and rejects active/pending-review references; initial release does not force-delete published history.

Clean Dolt merges remain subject to application rules. Two independent edits to different fields can merge mechanically yet violate a cross-row invariant. Treat that outcome as review-required and keep the valid live head. Deletes, delete/update conflicts, schema conflicts and legacy-ID collisions are included.

7. Server and plugin interfaces

Implement a typed federation client and equivalent REST, MCP and REPL surfaces; every first-party plugin exposes the same contract through its existing audited wrapper. New names below are planned interfaces, not existing capabilities:

- `federation.status(projectId, branch)` returns local head, published head as last observed, observed time, dirty state, ahead/behind counts, last commit time, waiting-local-work age, incoming target, connectivity, review count and operation status. `branch.list` shows the same freshness indicators. Flag stale branches at a configurable threshold; never automatically delete or rebase them. Never call a stale observation globally synchronized.

- `federation.diff(projectId, branch, from, to)` and `federation.history(...)` support preview without mutation.
- `federation.commit(projectId, branch, expectedHead, expectedWorkingSet, leaseToken, message, requestId)` creates a local version commit and pending publication intent. Reply distinguishes `committed_locally` from `published`.
- `federation.sync(projectId, branch, requestId)` schedules/wakes work and returns OperationId; `federation.operation.get(operationId)` returns durable status. Cancellation stops waiting/work safely without claiming already-published commits were undone.
- `federation.branch.list/create/select/merge` use explicit project/branch contexts and expected heads. Branch selection is session-scoped and survives plugin reconnect only after server revalidation.
- `federation.conflicts.list/get/preview/resolve` retain the full review evidence and reject stale input.
- Administrative `federation.enroll`, `subscriptions`, `control.poll`, `control.report`, `publication.accept`, `migration.plan/run/resume/verify/cutover/status` are permission-separated from ordinary editing.
- `federation.maintenance.preflight`, `federation.maintenance.attest` and `federation.maintenance.status` expose the same permission-separated maintenance service to deployment tooling and plugins. They enumerate drafts/branches, bind readiness to a maintenance epoch and inventory fingerprint, and report blockers before a binary swap.

Use existing normalized error envelopes with stable codes: `head_changed`, `working_set_changed`, `branch_busy`, `waiting_local_work`, `needs_review`, `incompatible_schema`, `unsupported_contract`, `unauthorized_project`, `lease_lost`, `remote_unavailable`, `migration_verification_failed`. Public DTOs and APIs require XMLDocs and OpenAPI documentation. Plugin bootstrap advertises capabilities and server contract version; old plugins fail clearly on unsupported commands without falling back to arbitrary SQL or shell access. Local request authorization uses existing local identities. Hub requests use the existing IdentityServer with per-node client credentials and project scopes; revocation is an IdentityServer action, validated by gateway introspection. It does not erase offline work. See section 13 and TR-MCP-FEDERATION-012.

Audit identifies the real agent/user and records preview, commit, publication, incoming acceptance, review, migration and source references. Replicating data does not execute original business commands again. GitHub actions, email and other external effects require their existing explicit commands and durable effect IDs; importing an effect record never sends it again.

Commit preview runs the existing sanitizer and selected secret-detection facility over promoted snapshots and the versioned diff; detected secrets block commit. Deleting or reverting a record does not erase earlier versions. A disclosure incident requires credential rotation, removal/revert of live content and, only under separate explicit operator approval, coordinated history rewriting plus re-cloning all subscribers. Normal synchronization never force-pushes as an incident shortcut. Ph0 locks the detector and history-retention policy; do not claim existing sanitization finds every possible secret.

8. Fourth database provider and schema migration

Current source evidence: `McpDatabaseProviderFactory` registers SQLite/PostgreSQL/SQL Server strategies; `McpDatabaseProviderKind` and `McpDbContextFactory` need a Dolt case. Storage targets net10.0 and central EF Core packages are 10.0.0. A .NET 9 example is not proof of EF10 compatibility.

Add canonical provider `dolt`, `DoltMcpDatabaseProviderStrategy`, configuration validation, design-time connection resolution, runtime health/provider diagnostics, and dedicated `QBrainAi.Storage.DoltMigrations` assembly with its own snapshot and baseline. Do not run SQL Server/PostgreSQL/SQLite migration SQL on Dolt. Preserve

existing providers, names, defaults and historical migration files; additive migrations are permitted where the approved context manifest requires. Add Dolt to package/build/deployment/test discovery and database administration UI/CLI selections. Runtime and design-time model options must match.

Ph0 selects a maintained EF10-compatible MySQL provider after executing real schema/query/transaction tests against a pinned Dolt image. Do not silently downgrade EF, install a prerelease provider, or write a bespoke EF provider if compatibility fails; report concrete alternatives for the operator. Test collations/case sensitivity, maximum index/key lengths, decimal precision, UTC timestamps, GUID/binary/text/JSON mappings, identity generation, optimistic concurrency, FK/delete semantics, raw SQL, query translation, migrations and parameterization. SQLite-specific search SQL must use a provider-compatible derived-index path rather than fail on Dolt.

Project-versioned and control contexts have separate migration histories. **Never run EF migrations at startup against a versioned project database.** Only the hub maintenance role applies project migrations on `maintenance/schema—<version>`, commits schema and migration-history rows together, validates and publishes the schema commit. Subscribers obtain schema changes through history exchange after their binaries advertise support, never by independently executing the same migration. Worker-to-hub protocol versions are negotiated separately from plugin capabilities; unsupported contracts return `unsupported_contract`.

Use compatible expand changes first. Contract changes wait for every enrolled subscriber/branch covered by the upgrade to acknowledge, or for explicit de-enrollment and a recovery plan for its unpublished work. An offline node cannot be magically quiesced: it may continue its supported local work, while the hub refuses incompatible publication. After binary upgrade, old-schema unpublished commits and feature branches merge the designated schema head through normal review and validation. Ph0 proves this path on representative schema changes. Do not point an old binary at an unsupported schema or silently discard old-schema commits. Control/standalone databases retain their ordinary controlled migration path.

Choose the **pre-upgrade draft-disposition policy** for this release: before replacing the binary, enumerate every local project branch, require drafts to be explicitly committed or otherwise resolved by their owner, and fetch the designated schema head. A dirty branch blocks the upgrade without touching its draft; no automatic commit/discard. Each branch stays closed to domain writes until the local sync worker merges the designated schema head into it (history merge, never local EF migration execution) and validation passes. A catch-up conflict opens needs_review for that branch only; other integrated branches reopen. The new EF model does not serve an unsupported old-schema branch; history/diff/review use the schema-aware maintenance surface. Retain old binaries/backups until verification. Unpublished commits survive in history; this policy does not assume simultaneous N/N-1 EF model compatibility.

Contract readiness is an authenticated node-attested inventory of **all** local branches, including unpublished ones, with each head and expand-schema ancestry, bound to a maintenance epoch and inventory fingerprint. A missing/offline attestation or a branch lacking the expand head blocks the contract step unless explicitly de-enrolled with recovery disposition. After attestation, branch creation/restore from an old schema is fenced until the transition finishes; historical branches may remain readable through compatible tooling but cannot become writable without catch-up. The hub cannot infer unpublished branch readiness from its remote alone.

9. Data migration tool

Deliver `qbrain database migrate` (planned CLI surface) with commands `plan`, `run`, `resume`, `verify`, `status`, `cutover` and `abort`. REST/plugin administration wraps the same application service. Source providers are SQLite, PostgreSQL and SQL Server; target is Dolt. Source access is read-only, and a source backup/snapshot is required. The command accepts secret references, never prints credentials, and does not write connection strings into receipts.

`plan` inventories source schema/version, projects, tables, relationships, provider-specific values, local-only data, legacy pending federation records and identity collisions. It emits an immutable manifest with MigrationId, source fingerprint, target/schema version, transformation rules and per-table counts. A dry run validates mappings and target compatibility without copying project rows or changing routing. Unmapped tables, lossy conversions and ambiguous legacy identity mappings block the run.

Migration has two explicit modes. **seed** is selected exactly once per ProjectId: it creates an isolated target, applies the initial project schema and imports the approved seed snapshot to establish C0. **join** clones that published C0, imports the joining host's mapped legacy differences onto `migration/<tag>/<MigrationId>` as an ordinary descendant commit, and uses the normal merge/review/publication path to join main. Never independently create three unrelated root histories and assume they will merge. The identity manifest accounts for each source row as already identical in C0, an explicitly mapped delta, a deliberate reviewed deletion or a local-only/excluded record. Absence on a joining source alone is not permission to delete a seed row.

Mapped deltas distinguish **new identity** inserts from **legacy divergence**, where the same mapped identity has different seed/join content. Every legacy divergence requires a recorded per-row review decision (keep seed, take join or manual value) in the immutable transformation manifest and audit **before the join commit is created**. This is required even when Dolt would fast-forward or merge cleanly: a common root created during migration does not prove that earlier independent edits were intentional overwrites. Unreviewed divergence blocks import/commit and is surfaced as needs_review. A reviewed manifest is versioned by hash; changing a decision creates a new manifest revision and cannot silently resume an earlier transformation.

Divergence is evaluated over B2-classified versioned columns after canonical normalization. Review decisions may be authored as explicitly scoped rules by entity and column class that expand to an enumerated per-row list in the manifest; there is no wildcard default. Operational timestamps/counters excluded by B2 do not generate artificial project-content conflicts.

Both modes create isolated targets and copy parent/child graphs in dependency order using stable keyset pages. Bulk copy is not a replay of business commands. Preserve IDs, relationships, timestamps, versions, soft deletes and complete sanitized session graphs through explicit promotion. Local audit remains local in control storage or a retained readable source archive according to the manifest. Credentials and source-host operational actions are not activated on the destination.

Use a quiesced source snapshot for initial release. No custom CDC or live dual-write is introduced. Each target batch transaction records its MigrationId/table/key range in a target-local journal; resume verifies that journal and source fingerprint before continuing. A copied batch with a lost acknowledgement is recognized, not inserted twice. Changed source snapshot or changed transformation manifest requires a new migration, not a blind resume.

`verify` compares counts and canonical typed row hashes by project/table, checks FK graphs and explicit excluded-row accounting, validates ID maps and sequence high-water marks, performs representative server reads, and verifies the initial Dolt commit. Canonical hashing specifies null, UTF-8 text, decimal scale/value handling, UTC precision and binary encoding; raw provider serialization is not a comparison standard. Verification failures name the project/table/key and preserve both sides.

`cutover` requires a successful verification receipt matching the exact frozen source and target heads; stops the affected workspace, switches its provider mapping atomically, rebuilds derived state and runs smoke checks before reopening writes. Abort before cutover leaves the source active and retains diagnostics. If writes have occurred after cutover, returning to the source requires an explicit reconciliation/export plan; never silently lose Dolt-only commits. Source databases/backups are retained, not dropped. Each node's divergent legacy data is inventoried independently; do not assume cloning the hub can replace it safely.

For join mode, verify binds to the immutable migration-branch commit and proves C0 is its ancestor. The destination workspace remains closed to writes until that commit has been integrated into local main through required review and validation. The cutover receipt records source snapshot, transformation manifest, migration commit, resulting local main head and published target observation. Verify both imported-source accounting at the migration commit and final-domain invariants at the main head; do not compare a post-merge head to a pre-merge verification hash as if they were identical.

Ph3 includes a verified Dolt-to-new-SQLite recovery export for TODOs and their child/link/alias graph. Keep original SQLite sources read-only. This is not a full-database reverse migration: non-TODO project changes need an explicit reconciliation decision. Ph8 cuts over one node first and requires a 48-hour soak before the second host; rollback cannot silently discard Dolt-only writes.

10. Deployment, decisions and evidence

Deployment target, not observed active configuration: LAB-OMARCHY is Linux at 10.42.0.214; PAYTON-LEGION2 is Windows at 10.42.0.143 on the 10.42.0.0/24 link. Both current source stores are SQLite, as clarified by the operator. SQL Server/PostgreSQL remain supported migration test sources. Repository paths are /home/sharpninja/github/McpServer and F:/github/McpServer. Use SSH LEGION2 as kingd through the existing authorized SSH configuration; no WSMan.

LAB uses version/digest-pinned dolthub/dolt-sql-server with persistent data. LEGION uses the checksum-pinned native Windows dolt.exe, supervised as a child of the existing McpServer Windows service; no assumed Linux container, WSL, Docker Desktop or new service-wrapper dependency. Plan data at F:/QBrainAi/Federation/{ProjectId}, binaries under F:/QBrainAi/tools/dolt/{version}, config outside source control. Graceful stop drains writes and checkpoints before stopping Dolt; forced termination is followed by journal/working-set recovery, never deletion of lock/data files. Windows ownership uses an exclusive FileShare.None handle plus a named mutex; Linux uses its supported exclusive owner lock. Ph0 B1-B12 execute on both real OSs, including restart, file locking and case/path behavior. Package/service details become pinned receipts before implementation, not inferred deployment claims.

Bind every SQL listener only to 127.0.0.1:3306 (and loopback IPv6 only if explicitly tested). Backend remotesapi is loopback-only on LAB port7448, hidden behind a proposed federation HTTPS gateway at 10.42.0.214:7447. Control, publication, audit and remote-facing routes are exposed only through that gateway; IdentityServer /connect is forwarded there to the existing local issuer. No new inbound federation listener is exposed on LEGION. Existing unrelated listeners are not changed by this plan. Verify both cross-host3306 probes fail and local health succeeds. Gateway route authorization must cover object/chunk endpoints and redirects as well as ref updates; no redirect may expose a token or bypass it.

Planned issuer name: QBrain Lab Federation Root CA. This is a proposed dedicated issuer, not a claim that a CA already exists. Ph0 records its owner, actual root/SPKI SHA256 fingerprints, leaf SANs (LAB-OMARCHY and10.42.0.214), expiry and rotation procedure before endpoint setup. Bootstrap distributes verified root and approved current/next SPKI pins over the established SSH trust path; TLS must validate chain, SAN and pin, with no trust-on-first-use or certificate-validation bypass. An existing approved lab CA may replace this proposed issuer only through a recorded decision and updated pins. No certificate/private key is generated during planning.

Credentials must never be printed or placed in command arguments, logs, chat, memory or commits. Use protected secret references/files and process-private injection. Receipts record executable, nonsecret arguments, versions, hashes and exit status; secret-bearing fields are omitted, not echoed then redacted. Dolt's documented DOLT_REMOTE_PASSWORD is supplied through ProcessStartInfo.Environment from the protected reference, never shell assignment text or argv. Restrict SQL logging and process-dump access. Canary scans cover receipts, logs, audit payloads and generated files on both hosts.

DEPLOY HOLD must be explicitly lifted by the operator for the exact Ph8 rollout. A passing test, plan approval or old HV verdict does not lift it. Backups cover Dolt objects/refs, local and hub operational stores, audit receipts, IdentityServer configuration/operational state and permission files. Restore starts with ingress disabled, uses a fresh random HubEpoch and reconciles every node's retained heads and audit receipt inventory; see TR-MCP-FEDERATION-006/008. A restored audit store requests resend by record IDs/hashes from permanent node ledgers. Never acknowledge data missing after restore. Decisions still needing gate evidence are the BDPv4 plan section 5 blocking outputs B1-B12; none is proven by this document. They are explicit feasibility outputs, not permission to invent replacements. No current performance, compatibility or implementation-ready 98/98 HV result is claimed.

Primary sources consulted 2026-11-10:

- Dolt storage engine and SQL architecture: versioning belongs to Dolt's own engine.
- SQL clients, EF sample, SQL procedures: existing integration surfaces, not this project's compatibility proof.
- Commit distinction, conflicts: SQL persistence differs from version commits; merges are relational.
- Remotes, SQL-server push behavior, branch permissions: remote transport and dirty-working-set/authorization boundaries require integration proof.
- Docker and server configuration: official deployment/configuration surfaces.

11. Audit flows to the hub independently of commits

Append audit locally with an outbox in the same operational transaction as its auditable operation where possible; an explicit recovery scanner covers cross-store crash windows. Deliver through authenticated POST /qbrainai/federation/audit/batches without waiting for a project commit. The hub ledger is append-only and unversioned. It does not replay business commands. Identity is (OriginNodeId,RecordId); exact retry returns the same payload-hash receipt; a changed payload under the same ID returns409 audit_id_conflict. Node and hub runtime roles cannot update/delete existing audit records. Hub indexing is a derived projection.

TODO audit history and generic audit history are migration-backfill domains. Verify source counts, typed payload hashes, origin identity and durable hub receipts. Retain permanent node ledgers even after acknowledgement. Audit payloads preserve the original author, event time, source identity and sanitization version; raw secrets never enter transferable payloads. Automatic session audit is delivered through this channel; intentionally promoted sanitized session snapshots may separately appear in project history. Full graph completeness, PlanFile/TodoId and sanitizer parity from TR-MCP-FED-SESSIONLOG-001 remain binding.

12. Identity, duplicate work and conflict visibility

Migration planning detects both same-ID/different-entity collisions and different-ID/same-work duplicates. Similarity compares normalized title, description and dependency context and emits evidence plus a score; it is a review aid, never authority to collapse records. Decisions keep both, alias to one canonical identity, or record an explicitly reviewed merge. Alias uniqueness is scoped by project, origin node, entity kind and legacy ID; aliases are permanent and included in recovery export. An ambiguous unqualified legacy ID produces ambiguous_legacy_id and asks for origin context. Historical text is immutable; UI/link resolvers resolve its IDs through the alias service.

Reserve human-assigned IDs such as IMPL-UNITGREEN-001 at the hub using atomic uniqueness and idempotent requestId. Offline creation uses a generated tagged provisional ID; reconnect reserves the requested human ID or reports human_id_reserved for review. This preserves offline autonomy without pretending

disconnected nodes can globally reserve the same human label. Immutable generated IDs and old aliases remain usable.

Review visibility is immediate in server status, all plugins and a durable operator inbox. needs_review and waiting_local_work warn at15minutes, escalate at24hours and repeat no more than hourly. These are local UI/inbox notifications; external messaging requires its existing explicit authorization. Identical edits and mechanically independent cells may merge after domain validation. TODO priority/status/completion, conflicting title/description, dependency deletion and delete/update disagreements require review; no time-based winner. One blocked branch cannot stop another branch/project's publication. Later commits dependent on a blocked branch remain local or move to an explicitly created branch; there is no hidden cherry-pick or discard.

13. Existing IdentityServer is the authority

Fold MCP-FEDIDENTITY-001 into Ph2/Ph5/Ph7: federation hosts manage users through the integrated Duende IdentityServer; each node is single-user and its machine client is bound to that authenticated owner. Client credentials prove the machine, not the human. Operator-authorized enrollment supplies the owner binding. Provision fed.node.{NodeId} clients and project/action scopes; do not reuse the static sample mcp-agent client as a fleet identity.

Use reference access tokens for network federation so revocation is an IdentityServer action: disable the client and remove its persisted reference grants. Control/publication/enrollment/audit APIs validate token activity, audience, project scope and current branch membership. No positive introspection cache; recheck at the public-ref commit boundary. Existing offline local identity authorization is separate and never waits for remote introspection.

Dolt remotesapi does not automatically understand these bearer tokens. The gateway is the authentication boundary. The planned remote relay accepts the node reference token through the Dolt client's supported protected credential channel, introspects it and translates to backend-only credentials; backend SQL/remotesapi never receives a node's unrestricted authority. Prove actual remotesapi protocol, HTTP/2, credential forwarding, chunk endpoints, redirect containment and candidate-repository isolation in Ph0. If a supported proxy/client integration cannot enforce this, stop that gate and present a supported alternative; do not expose a raw remote or invent a new object-transfer algorithm.

14. Current contracts and scope coverage

Technical requirements own schemas, endpoint contracts, scopes and stable error codes. Testing requirements own Given/When/Then expectations. The current-scope coverage index identifies applicable requirements recovered from the historical gap check and binds them to the new design. Only current in-scope contracts govern implementation; original source copies are audit evidence.

Additional primary references, consulted 2026-11-10: Dolt remote authentication, Duende reference tokens, Duende revocation, Structurizr DSL. Version-specific behavior still requires the pinned Ph0 evidence; newer documentation is not proof of compatibility with the repository's Duende7.4 packages.

Dolt federation: BDPv4 implementation plan

Author: Codex. Date: 2026-11-10. Status: proposed planning baseline; implementation readiness remains gated by Ph0 evidence. Independent review results are retained under docs/receipts/hv. No implementation, package installation, schema migration, data migration or federation activation has occurred.

Architecture authority: Dolt federation architecture. Workflow evidence and requirement inventory: three-system walkthrough. This plan replaces the old forwarding/read-switch execution sequence for versioned federation. Retain the October and earlier November drafts as historical evidence, not competing instructions.

1. Scope and acceptance

LAB-OMARCHY and two nodes must independently edit and commit the same logical project's data, exchange committed histories through the hub, preserve uncommitted work and branches, and review conflicts without pull requests. Add Dolt as a fourth supported database and provide a safe migration tool from SQLite, PostgreSQL and SQL Server. Existing providers and explicit DirectProxy remain supported. The second node is the isolated FED-ATLAS-B test peer; real-host acceptance uses LAB and LEGION, with the three-peer fixture provisioned in Ph0.

Success is demonstrated by every workflow U01-U15, every linked FR/TR/TEST criterion, all supported provider regressions and three-system restart/recovery tests. An empty control table is insufficient proof: compare accepted commit ancestry, project data and branch state, including pending drafts/reviews. Source-code Git, arbitrary file synchronization, PRs, automatic hub election, live cross-provider CDC and concurrent independent schema migrations are outside this release.

2. Fastest dependency sequence

Phase identifiers are Ph0 through Ph8; they do not denote priority levels.

1. Ph0: prove Dolt/EF10/provider/remote fit before investing in integration.
2. Ph1: add fourth provider and separate schema migration assembly; preserve original providers.
3. Ph2: split project-versioned and operational storage, branch-scoped access and editing ownership.
4. Ph3: deliver seed migration, audit-backfill preparation and verified TODO recovery export; keep targets isolated.
5. Ph4: expose explicit commits, branch/history/diff and local publication intent.
6. Ph5: implement enrollment, guarded remote publication and the hub table/poller.
7. Ph6: implement safe receiving, reviewed merges, join migration and end-to-end crash/restore recovery.
8. Ph7: expose complete client/MCP/REPL/plugin workflows and user-visible status.
9. Ph8: run two-real-host acceptance plus a named three-logical-peer fixture, then prepare a staged lab cutover under DEPLOY HOLD.

Prepare test fixtures, DTO/API documentation and plugin contract fixtures during earlier phases where independent, but do not move unfinished implementation past a Byrd gate. Provider feasibility, not old forwarding fixes, is now the critical path. Reuse existing identity, diagnostics, provider abstractions, audit and typed clients; replace forwarding/command-replay paths only for versioned projects.

3. Source-control and donor handling

Create an isolated integration worktree from the current accepted develop when implementation is authorized. Suggested branch `integration/federation-dolt-20261110`; use separate small phase PRs against it. Preserve all existing worktrees and unrelated working-tree edits. Do not create or clean worktrees merely to write this plan.

Reuse existing changes only when they satisfy an in-scope security, identity, path or regression requirement. Verify their current diff and tests before porting; unrelated worktrees and superseded forwarding machinery are not dependencies.

4. Byrd gate contract

For each small slice, first define the acceptance cases, fixtures and negative controls. Prove tests pass against controlled mocks and fail against deliberately wrong behavior. Then run the same contract against the missing/incorrect real implementation to record red, implement the smallest behavior, and refactor to green. Mocks must not simply return the expected result without testing the orchestration boundary.

Use targeted `./build.ps1 TestGroups --test-groups '<project>:<class>' --test-run-id '<phase-slice>'` after changes. At each phase exit run the entire current-plus-prior unit suite once with `./build.ps1 Test`, requiring zero failures and zero skips. `./build.ps1 Compile` and `./build.ps1 ValidateTraceability` are mandatory gate commands. Add real-container integration tests where a mock cannot prove SQL, migration, locking or remote semantics. Do not broaden/repeat a passing gate without changes or new evidence requiring it. Receipts include SHA/tree state, image/package versions, commands, TRX, fail/skip counts, AC mapping and negative controls.

The proposed test-class names below are implementation targets, not claims they exist today. Register new projects/classes in Nuke discovery as part of their owning phase. Do not add skipped placeholders for later phases; keep deferred coverage in requirements/TODO state.

Retain the operator's complete phase-exit gate on both LAB and LEGION: Nuke Test, MigrationIntegrationTests, the full Build.Tests suite, and ValidateTraceability at the same accepted head. Test currently excludes Build.Tests; do not infer its coverage from a green Test receipt. Ph0 records the complete discovered Build.Tests class inventory and its Nuke TestGroups invocation; every listed class and test must execute, with zero failures and zero skips. MigrationIntegrationTests and the new TestFederationProviders suite are complementary and neither replaces the other. Keep unit, integration and build-suite receipts distinct and run the full set once per final phase head, repeating only after changes or failed gates.

Add Nuke target `TestFederationProviders` with the planned command `./build.ps1 TestFederationProviders --providers sqlite,postgresql,sqlserver,dolt --test-run-id '<phase>-providers'`. It is not an existing command today. Ph0 records exact PostgreSQL/SQL Server/Dolt image versions and digests and the SQLite package version; Ph1 implements the target and refuses missing/unhealthy containers as a gate failure, never a skip. Every Ph1-Ph8 exit requires both the full unit gate and this real-provider/remote suite with zero failures and zero skips. Container-dependent classes are registered only in TestFederationProviders and excluded from Test by category at discovery, never runtime skipping. Receipts include image digests and environment readiness evidence.

5. Ph0: feasibility and exact dependency selection

Deliver a version-pinned compatibility receipt and license/dependency record for Dolt and the MySQL-protocol EF provider and driver selected under B1. Use current net10.0/EF10.0.0 as the baseline. Read the dependency compliance rules before additions. Existing Dolt EF samples establish possibility, not compatibility.

Prove on LAB Linux and LEGION native Windows against real databases (containers where appropriate): full current model creation; representative CRUD/LINQ/raw SQL; concurrency tokens; transaction rollback; decimal/date/text/index/FK mappings; migrations; branch connection isolation; explicit SQL commit versus Dolt commit; merge conflicts/constraints; clean and dirty remote pushes; lost responses; authenticated project isolation; inability to bypass guarded publication through raw remote requests. Verify canonical-ref advancement and remote recovery under process death. Compare candidate clean merges against domain validators.

Classes: `DoltProviderCompatibilityTests`, `DoltRemoteAuthorizationTests`, `DoltBranchIsolationTests`, `DoltCommitBoundaryTests`. Red conditions: unsupported mappings/query translation, ordinary SaveChanges creating history, pool branch leakage, unauthorized ref mutation or missing recovery guarantees. Green: all named contracts pass on pinned versions, no ad hoc provider fork or custom merge engine.

Ph0 runs a disposable harness and records exact commands, digests, TRX and fail/skip counts. Ph1 carries the proven Ph0 contract assertions into maintained classes under TestFederationProviders and passes them at Ph1 exit. If Ph0 changes repository code, the full Test gate also applies at Ph0 exit; disposable prototypes do not become an undocumented production dependency.

Ph0 exit consists of the following numbered **blocking outputs**. The implementation owner produces each evidence record; the operator accepts it, with independent HV by an agent family other than the author, assessing substantive accuracy/completeness. Each record stores owner, status (unproven/proven/accepted), evidence paths and dependent phases. Initial status for all B1-B12 is unproven. Every B1-B12 receipt has separate LAB and LEGION evidence rows; Windows is not inferred from Linux. Ph0 verifies the current-scope coverage index and binds each applicable acceptance criterion and workflow to its implementation phase. All must be accepted before Ph1 production integration begins; Ph0 prototypes remain disposable.

1. B1: exact provider/driver/image versions, digests, licenses and all-four-provider fixture versions (Ph1-Ph8).
2. B2: every entity, relationship, interactive/automatic writer and auto-updated column; session capture remains operational and only explicit promoted snapshots become versioned (Ph2-Ph8).
3. B3: full standalone McpDbContext versus Dolt-only project/control context topology, additive migration chains and cross-database soft-reference validators (Ph1-Ph3).
4. B4: exact ID grammar per generated/human-assigned family, legacy compatibility, sort/display, parser/plugin/traceability changes and restored allocator behavior (Ph2-Ph8).
5. B5: existing IdentityServer client-per-node/owner binding, reference tokens and scopes, gateway introspection/revocation, backend-only Dolt credentials, enforceable remote publication permissions and credential containment, worker-only remote procedures, gated worker live-branch mutation path and rotation/revocation (Ph2/Ph5-Ph8).
6. B6: real merge/constraint/conflict-transaction semantics, retained review evidence, safe conflict persistence including whether `dolt_allow_commit_conflicts` is needed, and unrelated-history behavior. Never enable unresolved conflicts on live project branches (Ph4-Ph6).
7. B7: single-author schema distribution, startup migration exclusion, expand/contract rules, mandatory pre-binary-swap draft disposition, all-local-branch attestation/fencing, offline old-schema commits and feature catch-up, worker protocol negotiation (Ph1/Ph3/Ph5/Ph6/Ph8).
8. B8: seed/join/recovery-export manifest, audit backfill, duplicate detection and permanent aliases, one common root, mandatory legacy-divergence review over B2-classified versioned columns after canonical normalization; reviewed scoped rules may expand to enumerated per-row decisions with no wildcard default. Include identity/deletion accounting, canonical hashes, immutable migration-commit plus final-main verification and closed-until-integrated cutover (Ph3 seed/export, Ph6 join, Ph8 acceptance).
9. B9: single writer ownership, write gate/fence transitions including every worker live-branch mutation, working-set hash API, requestId commit trailers and crash reconciliation (Ph2/Ph4-Ph6).

10. B10: measured fixture-scale storage/latency/history growth, resource budgets and explicit operating limits; no unsupported throughput/SLA claim (Ph1/Ph5/Ph8).
11. B11: polling/retry/stale-branch and15minute/24hour review-age thresholds, subscription-state schema and monotonic generation/epoch semantics (Ph5/Ph7).
12. B12: local/remote candidate/review refs, GC/retention, hub-restore epoch/reconciliation, network loopback isolation, named CA/pins and secret-safe receipts, plus secret-detection and disclosure response (Ph4-Ph8).

If a maintained EF10-compatible provider is unavailable, report concrete options for approval; do not downgrade the repository silently. This plan is executable through Ph0 but is not implementation-ready for later slices until these evidence-dependent choices are locked.

6. Ph1: fourth provider and schema migrations

Requirements: FR-MCP-186. Extend `McpDatabaseProviderKind`, `McpDatabaseProviderFactory`, `McpDbContextFactory`, runtime options/configuration validation, health output and setup tooling with canonical `dolt`. Add `DoltMcpDatabaseProviderStrategy` and `QBrainAi.Storage.DoltMigrations`, with provider-specific snapshot/baseline. Never edit existing migrations; add migrations only where B3 requires. Include provider selections in build/package/admin surfaces. Exclude versioned project databases from automatic startup migration; standalone/control contexts retain their approved migration path.

Classes: `DoltProviderResolutionTests`, `DoltMigrationTests`, `FourProviderContractTests`. Include VersionedProjectNotMigratedAtStartup in this owning phase. First prove unknown-provider rejection, unchanged defaults and distinct migration assembly selection. Real-provider tests cover empty database creation, baseline-to-next upgrade, interrupted startup/retry, idempotent reapplication, schema drift detection and application CRUD after upgrade. Test all original providers for regression. No production data copy in this phase.

7. Ph2: project routing, data boundaries and editing ownership

Requirements: FR-MCP-174,176,177,182,184,185,189. Add immutable ProjectId/local-path enrollment mapping, per-project provider selection, project and control contexts, full table classification, branch-scoped DbContext/query/index keys, local ID allocation and fenced edit leases. A versioned project's ordinary requests must never enter LocalProxy forwarding. Preserve DirectProxy behavior for explicit legacy configuration; no ambiguous combination on one project.

Fold MCP-FEDIDENTITY-001 owner binding and user-management requirements into this phase. Audit/outbox storage remains unversioned, but its delivery obligation is hub-bound, not local-only. Define scoped permanent aliases and local provisional IDs; host paths and credentials remain local. Classes: `FederationLocalExecutionTests`, `ProjectBranchContextTests`, `FederationDataBoundaryTests`, `NodeTaggedIdentityTests`, `BranchEditLeaseTests`. Cases include BackgroundWritersDoNotDirtyVersionedBranch, SecondAgentLoggingNeedsNoLease, LegacyIdsRemainValid and LeaseLostDuringWriteRejected. Red: offline request waits for hub, cross-project row visible, secret or branchable audit in clone, duplicate IDs after branch/restart, pooled connection changes branch, or one agent commits another's work. Green: local read-after-write/restart, complete entity/writer classification, no branch/project leakage, safe expired-lease recovery and stable replicated IDs. Provider schema changes introduced here require migrations for each affected supported provider, not Dolt alone.

LeaseLostDuringWriteRejected means admission after expiry/fence change is rejected and takeover waits for an already-admitted transaction; it does not demand aborting a transaction legitimately holding the gate through completion.

8. Ph3: seed migration and TODO recovery export

Requirements: FR-MCP-187, FR-MCP-182/184, FR-MCP-AUDIT-001 and the reconciled backfill contracts. Implement the shared migration planner, immutable source manifest, duplicate-candidate review, permanent alias mapping, seed-only run/resume/verify/status/abort and frozen-head Dolt-to-new-SQLite TODO export. Audit backfill preparation accounts for every historical TODO/generic audit row; actual hub delivery/receipt binding joins Ph5. Ph3 never invokes publication or reviewed join functionality that does not yet exist.

Classes: MigrationPlannerTests, MigrationResumeTests, MigrationIdentityMappingTests, MigrationVerificationTests, ThreeProviderToDoltMigrationTests and TodoRecoveryExportTests. Reuse TEST-MCP-235's exact Given/When/Then cases. SeedPhaseRejectsJoin returns unsupported_capability with no writes when mode=join is requested. ThreeSourceProviderRoundTrip covers SQLite,PostgreSQL,SQL Server fixtures; both live lab sources are SQLite. DoltTodoExportPreservesNewWrites proves a post-cutover TODO and its child/link/alias graph survive export while retained source bytes remain unchanged. Same-work/different-ID candidates require review; never rewrite historical text references.

Migration acceptance also covers multi-project separation, soft-deleted rows, decimal/UTC/binary/Unicode edge cases, complete promoted session graphs, hub-bound audit, derived-index rebuild and inert imported operational records. Inject death after target batch commit before acknowledgement, before provider switch and after switch before reopening. Dry-run and failed verification must preserve source bytes and active routing. Ph6 applies the same matrix to joins; source accounting and reviewed-main verification are separate checks.

Exit: disposable seed migrations and verified recovery exports pass all four-provider gates. Keep source SQLite snapshots read-only. No live cutover, join commit, public-ref update or source deletion occurs. Join-specific cases remain planned for Ph6, not skipped tests in Ph3.

9. Ph4: explicit history commits and local publication intent

Requirements: FR-MCP-175,177,182,183,184,189. Implement status/diff/history/commit preview, idempotent explicit commit, branch lifecycle and local unpublished-head scanning. No save hook implicitly creates Dolt history. A hub request cannot auto-commit drafts. Protect preview fingerprints, author provenance and branch edit ownership.

Classes: `FederationCommitTests`, `FederationBranchLifecycleTests`, `PublicationIntentRecoveryTests`. Include CommitRetryReturnsSameHash, CommitPreviewBlocksSecret and StaleBranchReportedNotDeleted. Red: edits trigger sync, commit bundles unauthorized changes, restart loses an unreported commit, or fetched remote commits cause endless republishing. Green: durable explicit commit triggers exactly the needed sync intent, history reconstruction repairs crash windows without duplicate commits, feature publication does not merge main.

10. Ph5: hub control table and publication

Requirements: FR-MCP-176,178,179,181,185. Provision the hub shared remote separately from its local working copy. Implement enrollment and project/branch subscription, candidate upload/validation/acceptance, expected-head publication, SyncSubscription state-table migrations, polling/reporting, durable receipts, worker epochs and remote-head reconciliation. One row per subscription replaces command-message generations/leases/coalescing. Both nodes initiate communication; H's local worker uses the same authenticated gateway. Add existing IdentityServer clients/scopes, audit/batches ingestion with idempotent receipts, human-ID reservation, restore epoch and polling. Test AuditReachesHub, AuditRetryIdempotent, SimultaneousHumanIdReservation, SqlPortUnreachableAcrossLink and HubRestoreFromOlderBackup. No native remote endpoint is exposed to node credentials.

Classes: `FederationEnrollmentTests`, `FederationPublicationTests`, `FederationControlTableTests`, `FederationPollingTests`. Include LocalAppUserCannotPush, RevokedCredentialFailsFetchAndPublish and OldWorkerEpochCannotReportSuccess. Red: node can publish unauthorized branch/project, hub drafts block publication, polling marks success, reporting generation g loses desired g+1, missing state update strands a published head, or simultaneous publishers overwrite history. Green: all controls work against real remotes and all control-store providers; no force pushes; stale heads yield deterministic retries.

11. Ph6: incoming integration, review, join migration and recovery

Requirements: FR-MCP-178,180,181,186,188,189. Implement temporary integration branches, clean-merge validation, dirty-branch deferral, gated live-branch application, conflict review with head-bound resolution, recovery pins/receipts, maintenance preflight/attestation/status and derived-state refresh. Incoming data must not re-execute business commands or external effects. Include IncomingMergeCannotRaceAdmittedEdit, covering an isolated worker that must use the owning server's gate API.

Classes: `FederationMergeReviewTests`, `FederationDirtyWorkingSetTests`, `FederationRecoveryTests`, `FederationEffectIsolationTests`. Cases include all-three concurrent nonconflicting commits, same-cell conflict, delete/update, invalid clean merge, missing ancestry, stale review, hub/node restart at each state transition, disconnect during upload, lost acknowledgement, concurrent resolution and unavailable derived indexes. Include OldSchemaUnpublishedCommitMergesAfterUpgrade, FeatureBranchSchemaCatchUp, DirtyDraftBlocksBinaryUpgrade and ContractBlockedUntilLocalBranchesAttested. The dirty-draft case proves the attempted swap is blocked and preserves the draft before schema merge. Preserve valid live data and all history; never report synchronized on conflict, dirty deferment or stale observation. Test older-schema node quarantine separately from network failure.

After Ph4 commit/branch and Ph5 publication services exist, implement join mode here: clone C0, stage duplicate/legacy-divergence decisions, import J1, review/merge into M1, validate audit-backfill receipts and verify both J1 source accounting and M1 domain state before cutover readiness. Classes MigrationJoinTests and MigrationCutoverTests own MigrationJoinPreservesCommonAncestor, MigrationJoinDivergentRowRequiresReview and JoinCutoverRequiresReviewedMain. Ph3 did not claim these dependencies were available.

Add ReviewAgeVisible, UnrelatedPublicationContinues, SafeTodoMergeRules, AuditBackfillVerified and HubRestoreFromOlderBackup. Permit safe identical/independent-cell merges only under the explicit TODO policy. Preserve unresolved branch history and permit unrelated project/branch publication. The single writer sentence applies to live branches; worker integration refs use their narrower rights and every live move uses the owner's gate.

12. Ph7: server and plugin workflows

Requirements: FR-MCP-183 and every workflow's plugin surface. Add typed DTO/client methods, REST/MCP/REPL adapters and equivalent audited plugin wrappers for installed first-party agents. Inventory actual plugin repositories and versions at phase entry; do not edit another plugin through the wrong agent identity. Expose capabilities, project/branch context, status/diff/history/commit/sync, branches, review, audit delivery status, identity enrollment/owner management, human-ID reservation, aliases, admin migration/export commands and maintenance.preflight/attest/status as defined by the architecture and TRs. Surface review/wait ages immediately, with15minute warning and24hour escalation in the operator inbox; no external messaging is sent automatically.

Classes: `FederationClientContractTests`, `FederationMcpToolTests`, `FederationReplContractTests`, `FederationPluginConformanceTests`. Shared contract fixtures run against each plugin. Verify accurate local-committed/published/received status, unsupported-version errors, reconnect context revalidation, no implicit commits in turn hooks, real agent identity, normalized errors and safe cancellation. Human walkthrough uses the same service contracts as agents; no hidden raw-SQL path.

13. Ph8: two-host acceptance, three-peer fixture and held cutover

Build from the accepted integration SHA. Real-host acceptance requires LAB-OMARCHY(10.42.0.214/Linux) and PAYTON-LEGION2(10.42.0.143/Windows), both starting from retained SQLite copies. It does not depend on an unprovisioned third machine. Separately, Nuke provisions an ephemeral three-logical-peer fixture named FED-ATLAS-H, FED-ATLAS-A and FED-ATLAS-B with isolated databases, loopback dynamic ports and disposable roots under TestResults/{runId}; Ph0 proves that provisioning path on both OSs. These are test processes, never discovered hosts or production enrollment. Execute U01-U15; use H/A for real cross-link tests and the fixture for three-writer interleavings. Collect full requirements/AC/UC coverage and compare observed heads/rows/errors to the canonical GWT contracts.

Ph8 explicitly owns DeploymentPreflightBlocksDirtyOrUnattestedBranch, WindowsWriterLockAndRestart, HubRestoreFromOlderBackup, SqlPortUnreachableAcrossLink, UnknownCertificatePinRejected and ReceiptsAndLogsContainNoCredentials. All B1-B12 evidence must include LEGION. An absent third physical host is not a blocker; absent fixture evidence is a failed gate. Before live cutover, inventory and back up both current lab hosts' provider databases, identity/configuration, legacy queue/fanout/conflicts and unpublished data. Run the migration planner on every participating legacy project. Designate one seed per ProjectId; every other source uses the clone-plus-delta join path, preserving common ancestry. Resolve identity collisions and pending legacy operations explicitly; do not drop them or run both legacy replay and Dolt publication for the same project. Verify target snapshots before switching. Keep federation disabled until this gate passes and the concrete cutover is approved under the operator's deployment policy. Approval records the tested TODO-to-SQLite export boundary and the explicit disposition of non-TODO changes. Cut over one host first, observe a 48-hour soak with successful offline/reconnect, audit delivery and recovery-export checks, then authorize the second host.

DEPLOY HOLD remains active until the operator explicitly lifts it for this exact rollout. Plan approval, historical 98/98 or test success is not that release. Deploy through Nuke using the Linux container/native Windows package choices, protected configuration and persistent storage. Its deploy target calls maintenance preflight and refuses the binary swap on any dirty or unattested branch; it consumes the maintenance epoch/readiness receipt. SSH to LEGION2 as kingd using the configured authorized key. Enable only accepted projects; verify local operations before first publication, then enable subscriptions and polling. Cutover approval names planned join downtime from source quiescence through reviewed integration. Final acceptance includes one hub-authored commit and one commit from each node, both branch and conflict workflows, offline commits, and verified source retention. Stop after validation/reporting; do not extend into PR features or unrelated worktree cleanup.

14. Traceability, review and readiness

The current-scope coverage index identifies applicable requirements recovered from the gap check. The workflow inventory maps the 16 release contracts to phases, U01-U15 and 91 named acceptance cases. Current implementation contracts and acceptance includes the canonical schemas, endpoints, error codes and full Given/When/Then outcomes once in the PDF. Test names alone are not acceptance coverage. Keep existing IDs and update their current text rather than creating competing requirements. Each substantive review amendment is persisted through the MCP plugin, read back and exported before phase exit. Historical source copies and unrelated obligations are not required plan reading or phase-exit work.

Before a phase exits, every applicable AC must have executable evidence and every implemented workflow must satisfy its preconditions, success and failure paths. ValidateTraceability checks the FR/TR/TEST links. All amended criteria remain pending until the implementation passes the defined gates.

Independent HV by an agent family other than the author targets accuracy and completeness of at least 98 each on the exact current architecture, plan, workflow and contract packet. This revision awaits that review and Ph0 feasibility evidence. Record source/PDF hashes and findings separately from the plan. DEPLOY HOLD requires explicit operator release after acceptance; neither plan approval nor a review score releases it.

Three-system federation walkthrough and requirements

Author: Codex. Date: 2026-11-10. Proposed behavior for the new architecture, not a report of deployed behavior. H=LAB-OMARCHY, A=PAYTON-LEGION2, B=isolated test peer FED-ATLAS-B. Project Atlas and symbolic commit names C0/A1/B1/H1/M1 are examples, not real records or hashes. The hub has a local working copy and a separate shared committed-history remote.

Standard workflows

U01: enroll and start the same project

H registers Atlas with immutable ProjectId and an initial approved C0. A and B enroll with distinct immutable node tags, map their own filesystem paths to Atlas, receive authorized subscriptions and clone C0. Each can now read and edit locally. An existing nonempty node database is inventoried/migrated, never overwritten by the clone. Server: enrollment, project identity, initial snapshot, scoped credentials. Plugins: discover/select project and branch; show local/remote heads. Requirements: FR-MCP-176,182,185,187.

U02: node A edits, then commits

A edits a TODO. Its local draft survives restart, but H and B remain at C0 and no sync is triggered. A previews and explicitly commits A1; this wakes synchronization. After validated publication, the hub remote is A1 and H/B receive control requests. Their workers fetch and accept A1 on clean branches; their local reads now include the TODO. Server: explicit commits, publication and control generation. Plugins: diff/commit/status; distinguish local commit from publication. Requirements: FR-MCP-174,175,178,179,183,189.

U03: work directly on the hub

H edits in its local working copy while A and B continue independently. H commits H1 and its normal worker publishes it to the shared remote; A/B receive requests. H's uncommitted draft before H1 neither travels nor prevents A/B publication because the hub remote is a separate repository. Requirements: FR-MCP-174,175,178,184.

U04: simultaneous clean edits on all three systems

Starting at C0, A commits a title edit A1, B commits an unrelated description edit B1, and H commits a separate requirement H1. A publishes first. B fetches A1, Dolt merges A1+B1 into M1, validation passes and M1 publishes. H fetches M1, merges H1 into M2 and publishes. A/B accept M2. Every original commit remains an ancestor. A publication race returns head_changed and repeats fetch/merge; no force push. This assumes nonconflicting cells and valid domain relationships. Requirements: FR-MCP-178,181,188.

U05: conflicting edits to the same field

A and B both change one requirement's priority from the same base to different values and commit independently. A publishes A1. B receives A1 while holding B1; merge reports a conflict. B retains its last valid local state, and H/A retain A1. B's plugin displays base/local/incoming values and requests review. A reviewed resolution creates

M1 including both histories, passes validation and publishes; H/A then accept it. Changed heads invalidate an old resolution. Delete/update and schema conflicts use the same visible review boundary. Requirements: FR-MCP-180,181,183.

U06: hub unreachable, then reconnect

The A/B-to-hub link fails; H retains local access to its own gateway and remote. H, A and B still execute local business operations and make H1/A1/B1 commits. H may publish H1 locally while A/B record pending publication; they do not return queued-only results for local business writes. On reconnect A merges H1+A1 into M1 and publishes, then B merges M1+B1 into M2 and publishes; H/A catch up to M2. They authenticate, reconcile remote heads and process requests plus their own unpublished commits. Merge/review follows U04/U05. An empty hub queue does not strand local commits. No automatic hub failover is claimed. Requirements: FR-MCP-174,175,178,179,181,185.

U07: incoming commit while node B has a draft

B has uncommitted edits at C0. A publishes A1; B polls and fetches history, but records waiting_local_work instead of committing, discarding or overwriting its draft. B's user finishes the draft and explicitly commits B1 (or separately chooses another authorized disposition). Then reconciliation proceeds. Fetch succeeded does not mean the branch was synchronized. Requirements: FR-MCP-175,180,183,189.

U08: divergent feature work and deliberate merge

A creates feature/search from C0 and commits F1. Publication shares feature/search; main remains C0 on all systems. B selects feature/search to inspect or continue it and commits F2. H can review the branch data. An authorized explicit merge into main creates M1 and triggers normal publication. Branch selection is per session, never a process-wide switch. No PR is created. Requirements: FR-MCP-177,180,183,189.

U09: lost response or crash during synchronization

A publishes A1 but loses the success response. H has A1 and unsatisfied subscription state. A restarts, checks remote ancestry and its request receipt, and recognizes publication without creating A2 or repeating an external business action. A missed hub state update is repaired by head-generation reconciliation. A new worker epoch rejects stale reports from the old worker. Reporting generation g must not clear desired g+1. Requirements: FR-MCP-179,181,188.

U10: migrate an existing project

Both real lab stores are SQLite. The migration tool additionally supports PostgreSQL and SQL Server fixture sources. It plans those sources, inventories local-only versus shared data, detects legacy-ID collisions and checks the Dolt schema. Ph3 delivers seed and TODO recovery export; Ph6 adds join only after branch/commit/publication/review exist. Exactly one seed establishes C0 per project. Other nodes clone C0 and import their frozen legacy differences on migration branches descended from C0, then merge/review them normally. Verification accounts for source rows already in C0, mapped deltas and explicitly reviewed exclusions/deletions. Include TODO/generic audit history in the independent hub backfill and verify every durable receipt. Detect same-work/different-ID duplicates for review and preserve permanent origin-scoped aliases, without rewriting historical text. A failed verification leaves the original provider active. No unrelated root histories or implicit deletion of seed rows are allowed. Requirements: FR-MCP-182,184,186,187.

Same-identity rows with different legacy contents require a recorded keep-seed/take-join/manual-value review before the join commit, even if Dolt would merge cleanly. Verify the immutable migration commit and the reviewed main result; keep destination writes closed until integration and verification finish.

U11: node running an older schema/application

H's maintenance role publishes a supported schema upgrade. B advertises an older capability and records incompatible_schema; it does not apply it or claim to be current. B may continue supported local work while publication is held. After preflight and binary upgrade, B merges S1 with its retained unpublished B1 to form M1, validates and publishes M1, then H/A accept M1. Both B1 and S1 remain ancestors; B does not simply replace B1 with S1. Each feature branch catches up separately through review/validation. Nodes never independently run EF migrations on versioned project startup. Incompatible contract changes wait for subscriber acknowledgement or explicit de-enrollment/recovery. Requirements: FR-MCP-181,183,186.

Before swapping B's binary, its owners explicitly commit or resolve all drafts and the schema head must be available locally; otherwise upgrade waits without touching drafts. B attests every local branch, including unpublished ones. Maintenance preflight gates deployment. Each branch reopens only after its local worker merges the designated schema history and validation passes; a catch-up conflict blocks that branch only. The new EF model never edits an unsupported dirty working set or independently executes project migrations.

U12: two agents on one host

Two plugins select Atlas/main on A. Only the editing lease owner may mutate its working set. The other agent reads or selects a separate branch. The first agent's commit preview binds to its expected working-set fingerprint; takeover cannot silently absorb its draft. Plugin reconnect revalidates the lease and context. Requirements: FR-MCP-177,183,189.

U13: audit reaches the hub without a project commit

Precondition: A is enrolled in Atlas; main=C0 and A has authorized local audit storage. A edits T1 while offline, appending immutable audit E1 and its outbox entry. On reconnect, audit/batches sends E1 through the IdentityServer gateway, independent of Dolt commits. The hub stores it append-only and returns a hash-bound receipt. A lost receipt causes retry, not duplication; the node retains E1 permanently. Postcondition: one E1 at each origin/hub ledger, main and remote still C0, local draft preserved. Errors: invalid_token, unauthorized_project, audit_id_conflict. Requirements: FR-MCP-AUDIT-001, FR-MCP-184/185/188. Ph2/Ph3/Ph5/Ph7.

U14: duplicate work, permanent aliases and human IDs

Precondition: frozen LAB TODO-7 and LEGION TODO-91 describe the same task. Planning emits similarity evidence; it does not infer identity automatically. An authorized review chooses keep-both or one canonical entity with both scoped IDs retained as aliases. Historical commits, PRs, session logs, memories and documents remain unchanged. For new human labels, two concurrent reservations of IMPL-UNITGREEN-001 yield one owner and one human_id_reserved response. Offline work uses a tagged provisional ID until reservation succeeds. Postcondition: every old reference remains resolvable with origin context, no unreviewed row disappears, and an ambiguous unqualified ID reports ambiguous_legacy_id. Requirements: FR-MCP-182/187. Ph2/Ph3/Ph5/Ph6.

U15: restore a hub older than a node

Precondition: A1 was published under epoch E1/generation90 and remains on A/B, but the backup contains C0/generation40. Restore keeps ingress disabled, installs a fresh E2 and advertises only verified restored heads. A node's E1 report receives hub_epoch_changed; it retains A1. The hub requests candidate history and validates ancestry before republishing A1 under E2. Audit inventories reconcile independently and permanent node ledgers resend missing records. Postcondition: no force push or forced node rewind, all recovered history preserved, receipts refer to E2 and no missing audit row is falsely acknowledged. Requirements: FR-MCP-179/181/184/188. Ph5/Ph6/Ph8.

UC preconditions, success and exception coverage

U01 starts with authenticated owner/node enrollment and empty target; succeeds with shared C0 and distinct local paths; nonempty/foreign target is denied. U02 starts with editable C0; succeeds with explicit A1 and eventual peer acceptance; draft or failed publication remains local. U03 is U02 on H with a separate shared remote. U04 starts at common C0 and three distinct edits; succeeds at M2 preserving H1/A1/B1/M1; conflict takes U05. U05 starts with same-base conflicting commits; succeeds only at an authorized valid merge; stale preview is rejected. U06 starts with enrolled peers and link loss; succeeds after M1/M2 reconciliation without queued-only local writes. U07 starts with a dirty B branch; succeeds only after the owner explicitly disposes the draft and merge validates; waiting age remains visible. U08 starts with C0 and authorized feature creation; sharing leaves main unchanged and explicit merge creates M1. U09 starts with an accepted commit and lost reply; retry returns the same hash/receipt without replay. U10 starts with retained frozen sources; success requires seed/join accounting, reviewed aliases, audit receipts and verified main; any failed verify blocks cutover. U11 starts with an unsupported incoming schema and unpublished work; success preserves B1 plus S1 ancestry after preflight and M1 validation; dirty/unattested branches block upgrade. U12 starts with agent1's edit lease; agent2 reads or selects another branch; no commit captures agent1's draft. U13-U15 state their own preconditions/postconditions above. Exact row/head/error assertions are canonical TEST-MCP-222..237 criteria, not inferred from diagram labels.

New requirement and feature inventory

The IDs below are the intended canonical records; the registration receipt establishes whether persistence succeeded. Status is pending throughout. Each FR maps one-to-one in order to TR-MCP-FEDERATION-001..016 and TEST-MCP-222..237. This inventory is additive; conflicting legacy-mode descriptions require explicit reconciliation, not blanket deletion.

FR-MCP-174: autonomous local execution

Every node, including the hub, performs authorized local business operations and persists results without hub connectivity. Phase: Ph2. Workflows: U02,U03,U06. Tests: NodeWorksWithHubUnavailable; HubUsesSameLocalExecution; OfflineLocalReadAfterWrite.

FR-MCP-175: explicit version commits and publication boundary

Only an explicit version-history commit triggers publication; drafts and SQL saves remain local. Commit retries recover the same history identity. Phase: Ph4. Workflows: U02,U07,U09. Tests: SaveDoesNotPublish; VersionCommitTriggersSync; HubRequestDoesNotCommitDraft; OfflineCommitDiscoveredAfterRestart.

FR-MCP-176: logical project enrollment and initial clone

Nodes enroll a logical project under immutable shared identity while retaining host-local paths and refusing destructive clones. Phase: Ph2/Ph5. Workflows: U01. Tests: DifferentHostPathsSameProject; CloneRefusesNonemptyTarget; UnauthorizedProjectDenied.

FR-MCP-177: branches across server and plugin context

First-release project branches support isolated work, explicit sharing and reviewed merge; session selection never leaks between connections. Phase: Ph2/Ph4/Ph6. Workflows: U08,U12. Tests: PooledConnectionDoesNotLeakBranch; FeaturePublicationLeavesMainUnchanged; BranchMergeRequiresExpectedHeads.

FR-MCP-178: bidirectional committed-history exchange

Exchange committed history bidirectionally through guarded publication, preserving every accepted ancestor and the hub independent working copy. Phase: Ph5/Ph6. Workflows: U03,U04,U08. Tests: ThreeWriterCleanConvergence; HubDraftDoesNotBlockRemote; ConcurrentPublishPreservesBothHistories.

FR-MCP-179: hub control table and polling

Use a durable hub subscription table with polling, generation and worker fencing, plus a fresh hub epoch after restore. Phase: Ph5. Workflows: U02,U06,U09,U15. Tests: PollReadDoesNotAcknowledge; AckOldGenerationPreservesNewRequest; MissedNotificationReconciles; HubRestoreFromOlderBackup.

FR-MCP-180: reviewed conflicts and valid merges

Conflicts and invalid merges preserve valid live data, require authorized review and remain visible with ageing while unrelated publication continues. Phase: Ph6/Ph7. Workflows: U05,U07,U08. Tests: SameCellConflictNeedsReview; DeleteUpdateNeedsReview; CleanMergeCanFailDomainValidation; StaleReviewRejected; ReviewAgeVisible; UnrelatedPublicationContinues; SafeTodoMergeRules.

FR-MCP-181: recovery without lost or repeated work

Recover synchronization, receipts and restored hub state without losing local work, inventing success or replaying business effects. Phase: Ph5/Ph6. Workflows: U06,U07,U09,U15. Tests: LostPushAckDoesNotDuplicateCommit; OldWorkerEpochCannotReportSuccess; DirtyBranchDefersApply; IncompatibleSchemaDoesNotApply.

FR-MCP-182: collision-resistant and preserved identity

Prevent generated-ID collisions, preserve permanent legacy aliases, detect duplicate work and atomically reserve human-assigned IDs. Phase: Ph2/Ph3/Ph5/Ph6. Workflows: U01,U10,U14. Tests: TwoNodesCreateDistinctIds; BranchRevertDoesNotReuseSequence; LegacyCollisionPreservesReferences; LegacyIdsRemainValid; DifferentIdsSameTodoDetected; SimultaneousHumanIdReservation; OfflineHumanIdPublicationDenied.

FR-MCP-183: first-party plugin federation contract

Expose consistent typed federation, review, identity, audit, migration and maintenance workflows through every first-party plugin. Phase: Ph7. Workflows: U01,U02,U05,U07,U08,U11,U12,U13,U14,U15. Tests: EveryPluginUsesSameContract; OldServerReportsUnsupportedCapability; ReconnectRevalidatesContext; StaleBranchReportedNotDeleted.

FR-MCP-184: portable data versus host-local state

Separate versioned project content from operational state while delivering permanent audit to the hub independently of commits and excluding secrets. Phase: Ph2/Ph3/Ph5/Ph7. Workflows: U10,U13. Tests: CloneOmitsSecrets; AuditReachesHub; AuditRetryIdempotent; AuditBackfillVerified; FullProjectSessionGraphPreserved; AcceptedHeadRebuildsBranchIndex; BackgroundWritersDoNotDirtyVersionedBranch; SecondAgentLoggingNeedsNoLease; CommitPreviewBlocksSecret; ReceiptsAndLogsContainNoCredentials.

FR-MCP-185: scoped authorization and offline behavior

Reuse integrated IdentityServer for single-user node ownership, per-node client credentials, project scopes and revocation; preserve authorized offline work. Phase: Ph0/Ph2/Ph5. Workflows: U01,U06,U13. Tests: RawRemoteCannotBypassPublicationPolicy; ForeignProjectCloneDenied; RevokedNodeCannotPublish; OfflineLocalAuthorizationStillApplies; LocalAppUserCannotPush; RevokedCredentialFailsFetchAndPublish; SqlPortUnreachableAcrossLink; UnknownCertificatePinRejected.

FR-MCP-186: Dolt fourth-provider support and schema migration

Support Dolt as the fourth provider with dedicated schema migration, controlled schema-history distribution, Windows/Linux lifecycle and original-provider regressions. Phase: Ph0/Ph1/Ph6/Ph8. Workflows: U11. Tests: FourProvidersResolve; DoltBaselineAndUpgrade; OriginalProviderMigrationsUnchanged; UnsupportedSchemaRejected; VersionedProjectNotMigratedAtStartup; OldSchemaUnpublishedCommitMergesAfterUpgrade; FeatureBranchSchemaCatchUp; DirtyDraftBlocksBinaryUpgrade; ContractBlockedUntilLocalBranchesAttested; DeploymentPreflightBlocksDirtyOrUnattestedBranch; WindowsWriterLockAndRestart.

FR-MCP-187: original-provider-to-Dolt migration tool

Migrate all three original providers using verified seed then reviewed join, permanent identities, audit backfill and a source-retaining TODO-to-SQLite recovery export. Phase: Ph3 seed/export; Ph6 join. Workflows: U10,U14. Tests: ThreeSourceProviderRoundTrip; ResumeAfterBatchCommit; ChangedSnapshotRejected; FailedVerifyCannotCutover; SourceRetainedAfterCutover; MigrationJoinPreservesCommonAncestor; MigrationJoinDivergentRowRequiresReview; JoinCutoverRequiresReviewedMain; DoltTodoExportPreservesNewWrites; SeedPhaseRejectsJoin.

FR-MCP-188: imported state does not repeat external effects

Imported project or audit state never replays external effects and general federation remains outside QuadBrain-only keyserver gating. Phase: Ph2/Ph6. Workflows: U04,U09,U13. Tests: ImportDoesNotSendExternalAction; RetryDoesNotRepeatEffect; ImportedOperationalRowsRemainInert; FederationBypassesQuadBrainKeyserver.

FR-MCP-189: safe shared-host editing and commit ownership

Enforce branch edit ownership and fenced live-write serialization so no agent or sync worker captures or overwrites another agent draft. Phase: Ph2/Ph4/Ph6. Workflows: U07,U12. Tests: CommitCannotCaptureOtherAgentDraft; ExpiredLeaseFenced; PreviewChangeRequiresReview; LeaseLostDuringWriteRejected; CommitRetryReturnsSameHash; IncomingMergeCannotRaceAdmittedEdit.

Planning boundaries

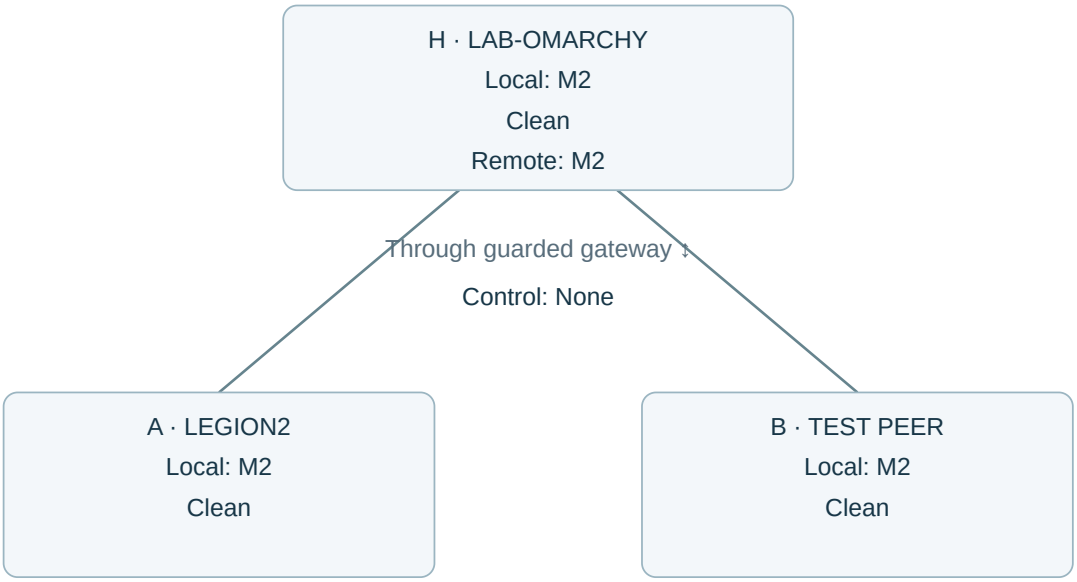
The operator requested Dolt support and the migration tool; no packages or deployments are authorized by this document alone. Exact provider/version compatibility and full entity partition remain Ph0 evidence gates. The table/poller/executor, plugin UX and validation are application work; the Dolt storage/history/merge algorithms are reused. PRs remain out of scope. Test names above are required behavior, not claims of existing or passing tests.

The new-contract slice retains its48 IDs and 16 mappings; the reconciliation register additionally covers legacy and cross-cutting requirements from both hosts. Each HV revision updates existing FR/TR/TEST records through the plugin and refreshes the export before Ph0 exit; no duplicate requirement set is created.

U04 · Three simultaneous commits

Atlas example · H: LAB-OMARCHY · A: PAYTON-LEGION2 · B: isolated FED-ATLAS-B test peer

Step 5 / 5

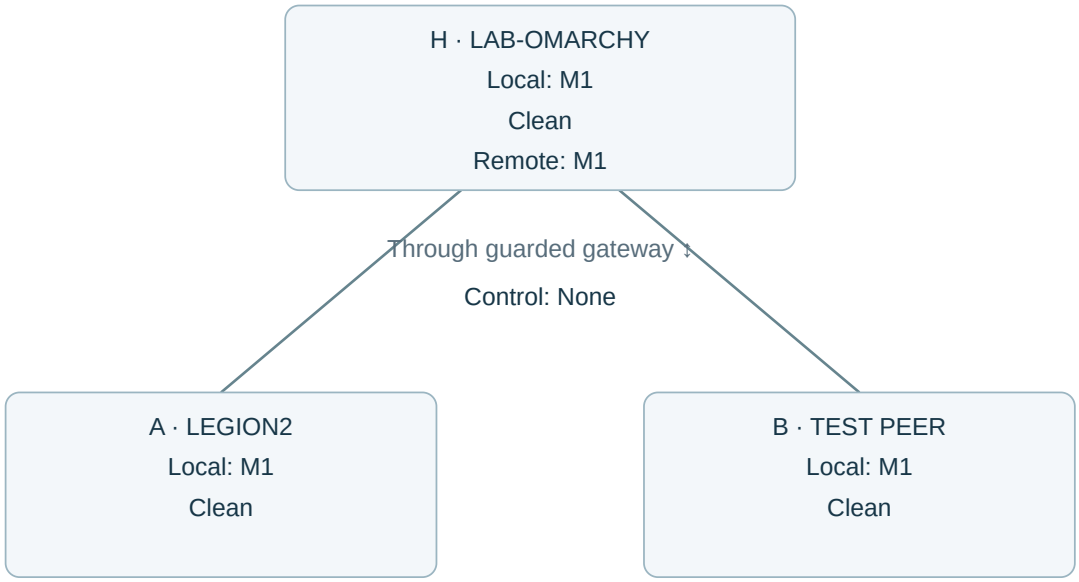


All accept M2; A1, B1 and H1 remain in its ancestry.

U05 · Same-field conflict and review

Atlas example · H: LAB-OMARCHY · A: PAYTON-LEGION2 · B: isolated FED-ATLAS-B test peer

Step 4 / 4



M1 publishes and propagates; both histories survive.

Current implementation contracts and acceptance

This is the current specification for the new federation release. Each technical contract and acceptance criterion appears once here, with its canonical IDs. These are planned acceptance obligations, not claims of implementation or passing tests. C0/A1/B1/H1/M1/M2/S1 refer to exact hashes captured in the fixture manifest. Historical source versions and superseded behavior are excluded.

1. autonomous local execution

FR-MCP-174 / TR-MCP-FEDERATION-001 / TEST-MCP-222. Phase: Ph2. Workflows: U02, U03, U06.

Every node, including the hub, performs authorized local business operations and persists results without hub connectivity.

Local request routing binds immutable ProjectId, NodeId and Branch to one DbContext and branch write gate. Dolt-mode domain requests never invoke LocalProxy forwarding or wait for the hub. Existing local authentication remains mandatory; federation reference-token introspection is not a dependency of local work. Result fields distinguish local persistence, version commit and publication.

- **NodeWorksWithHubUnavailable** (`ac-NodeWorksWithHubUnavailable`): Given A/main=C0 and the hub link is down, when A changes TODO T1.title from old to local, then the local API succeeds, a local reread returns local, and no hub request or history commit occurs.
- **HubUsesSameLocalExecution** (`ac-HubUsesSameLocalExecution`): Given H/main=C0 and the gateway is stopped, when H edits T1.title, then H persists it locally while remote/main remains C0.
- **OfflineLocalReadAfterWrite** (`ac-OfflineLocalReadAfterWrite`): Given the prior offline edit, when the application and Dolt restart, then T1.title remains local and main remains C0 until an explicit version commit.

2. explicit version commits and publication boundary

FR-MCP-175 / TR-MCP-FEDERATION-002 / TEST-MCP-223. Phase: Ph4. Workflows: U02, U07, U09.

Only an explicit version-history commit triggers publication; drafts and SQL saves remain local. Commit retries recover the same history identity.

POST /qbrainai/federation/projects/{projectId}/branches/{branch}/commits accepts requestId UUID, expectedHead, expectedWorkingSet, leaseFence, author and message. Success is 201 with commitHash and publicationState=committed_locally. Persist requestId in the Dolt commit trailer. Retry returns the original hash. Head/fingerprint/fence mismatch is HTTP409 with head_changed/working_set_changed/lease_lost. SaveChanges and control.poll never create version commits.

- **SaveDoesNotPublish** (`ac-SaveDoesNotPublish`): Given main=C0 and remote=C0, when SaveChanges persists T1.title=draft, then main and remote stay C0 and no publication intent is emitted.
- **VersionCommitTriggersSync** (`ac-VersionCommitTriggersSync`): Given that draft and a valid preview, when commit request R1 succeeds, then main=A1, the reply says committed_locally, and one recoverable publication intent targets A1.

- **HubRequestDoesNotCommitDraft** (`ac-HubRequestDoesNotCommitDraft`): Given B has a draft and desired head A1, when B polls, then outcome=waiting_local_work, B/main=C0 and the draft value is unchanged.
- **OfflineCommitDiscoveredAfterRestart** (`ac-OfflineCommitDiscoveredAfterRestart`): Given A1 committed while disconnected and a crash before intent persistence, when A restarts, then history scanning rediscovers A1 exactly once and publishes it after reconnect.

3. logical project enrollment and initial clone

FR-MCP-176 / TR-MCP-FEDERATION-003 / TEST-MCP-224. Phase: Ph2/Ph5. Workflows: U01.

Nodes enroll a logical project under immutable shared identity while retaining host-local paths and refusing destructive clones.

ProjectEnrollment stores ProjectId UUID, NodeId, ownerSubjectId, normalized local path, schemaVersion and subscriptions. POST /qbrainai/federation/enroll accepts an authenticated pre-provisioned node client plus operator-authorized owner binding. Body ProjectId must be allowed by token scope fed.project.{ProjectId}.enroll and durable membership. Paths stay host-local; no default-workspace fallback. A nonempty clone destination returns409 target_not_empty.

- **DifferentHostPathsSameProject** (`ac-DifferentHostPathsSameProject`): Given LAB path /home/sharpninja/github/McpServer and LEGION path F:/github/McpServer, when both enroll as ProjectId Atlas, then both reference Atlas and retain their distinct local paths.
- **CloneRefusesNonemptyTarget** (`ac-CloneRefusesNonemptyTarget`): Given target has TODO X, when clone is requested, then409 target_not_empty leaves X and all target files unchanged.
- **UnauthorizedProjectDenied** (`ac-UnauthorizedProjectDenied`): Given node A has only Atlas membership, when it enrolls or clones SecretProject, then403 unauthorized_project returns no rows, objects or project existence details.

4. branches across server and plugin context

FR-MCP-177 / TR-MCP-FEDERATION-004 / TEST-MCP-225. Phase: Ph2/Ph4/Ph6. Workflows: U08, U12.

First-release project branches support isolated work, explicit sharing and reviewed merge; session selection never leaks between connections.

Branch APIs create/list/select/merge using ProjectId, branch and sessionId. Each pooled connection is opened for an explicit branch and verified before use. Merge accepts expectedSourceHead and expectedTargetHead; stale values yield409 head_changed. Reserved refs upload/, integration/ and review/ are inaccessible to ordinary editing credentials. Branch deletion fails409 branch_busy while leases, reviews or recovery pins reference it.

- **PooledConnectionDoesNotLeakBranch** (`ac-PooledConnectionDoesNotLeakBranch`): Given main T1.title=main and feature/search T1.title=feature, when alternating pooled reads run100 times, then each returns only its selected value.
- **FeaturePublicationLeavesMainUnchanged** (`ac-FeaturePublicationLeavesMainUnchanged`): Given main=C0 and feature/search=F1, when F1 publishes, then remote feature=F1 and all main heads remain C0.
- **BranchMergeRequiresExpectedHeads** (`ac-BranchMergeRequiresExpectedHeads`): Given a preview for main=C0 and feature=F1 but feature advances to F2, when merge submits the old preview, then409 head_changed creates no commit and changes no ref.

5. bidirectional committed-history exchange

FR-MCP-178 / TR-MCP-FEDERATION-005 / TEST-MCP-226. Phase: Ph5/Ph6. Workflows: U03, U04, U08.

Exchange committed history bidirectionally through guarded publication, preserving every accepted ancestor and the hub independent working copy.

Node upload authority is limited to its candidate namespace in an isolated upload repository. POST /qbrainai/federation/publication/accept accepts projectId, branch, requestId, candidateHead, expectedPublishedHead and hubEpoch. Gateway validates authorization, ancestry, schema and domain invariants before the sole fenced publisher advances the public ref without force. Token validation occurs at the gateway; native Dolt remotesapi is loopback-only and never trusts a node bearer token directly. Transport/procedure enforcement is a Ph0 proof gate.

- **ThreeWriterCleanConvergence** (`ac-ThreeWriterCleanConvergence`): Given C0 contains T1.title=old,T1.description=old,T2.priority=low, when A1 changes title=A, B1 changes description=B and H1 changes T2.priority=high, then merges M1(A1,B1), M2(M1,H1) converge all heads to M2 with values A,B,high and all three commits in ancestry.
- **HubDraftDoesNotBlockRemote** (`ac-HubDraftDoesNotBlockRemote`): Given H has an uncommitted title draft at C0, when A publishes A1, then remote=A1, H remains C0 with its draft, and B can accept A1.
- **ConcurrentPublishPreservesBothHistories** (`ac-ConcurrentPublishPreservesBothHistories`): Given A1 and B1 descend from C0, when both accept requests expect C0, then one succeeds and the other receives409 head_changed; after merge M1 publishes, both A1 and B1 remain ancestors.

6. hub control table and polling

FR-MCP-179 / TR-MCP-FEDERATION-006 / TEST-MCP-227. Phase: Ph5. Workflows: U02, U06, U09, U15.

Use a durable hub subscription table with polling, generation and worker fencing, plus a fresh hub epoch after restore.

SyncSubscription key=(NodeId,ProjectId,Branch) contains HubEpoch UUID, PublishedHead, PublishedGeneration int64, DesiredGeneration int64, ReportedGeneration int64, WorkerEpoch int64, AcceptedHead, Outcome and timestamps. POST control/poll is read-only. POST control/report includes all epochs and observed generation; stale worker=409 stale_worker_epoch, stale hub=409 hub_epoch_changed. Poll every30s with20% jitter; commit/reconnect/on-demand wake immediately. Backoff1s doubling to60s. Generation ordering is valid only within HubEpoch. Restore generates a new non-reused random epoch before listeners reopen.

- **PollReadDoesNotAcknowledge** (`ac-PollReadDoesNotAcknowledge`): Given desired=7 and reported=6, when poll runs twice, then both responses request7 and stored reported remains6.
- **AckOldGenerationPreservesNewRequest** (`ac-AckOldGenerationPreservesNewRequest`): Given desired becomes8 while report7 is in flight, when report7 arrives, then reported=7, desired=8 and the subscription remains pending.
- **MissedNotificationReconciles** (`ac-MissedNotificationReconciles`): Given remote=A1 but control still names C0 after a crash, when reconciliation runs, then it records A1 and advances desired generation without producing another commit.
- **HubRestoreFromOlderBackup** (`ac-HubRestoreFromOlderBackup`): Given a node remembers epoch E1 generation90 and the hub restores a backup at generation40, when the hub reopens, then epoch E2 differs from E1; E1 reports fail409 hub_epoch_changed and no head is force-reset. Node A1 is retained until verified ancestry recovery republishes missing history.

7. reviewed conflicts and valid merges

FR-MCP-180 / TR-MCP-FEDERATION-007 / TEST-MCP-228. Phase: Ph6/Ph7. Workflows: U05, U07, U08.

Conflicts and invalid merges preserve valid live data, require authorized review and remain visible with ageing while unrelated publication continues.

Review record key=ReviewId UUID; fields ProjectId,Branch,baseHeads[],localHead,incomingHead,candidateRef,conflictingCells[],validationErrors[],createdUtc,lastNotifiedUtc,resolution,actor. GET conflicts and status expose it. Resolve binds exact heads and returns409 stale_review if changed. Safe TODO policy: identical concurrent values collapse; edits to different cells may merge only if invariants pass; duplicate immutable child IDs deduplicate only when bytes agree. Status, priority, title, description, dependency deletions and same-cell disagreements always require review; no timestamp last-writer-wins. Notify immediately in status and durable operator inbox; warn at15min, escalate at24h, repeat at most hourly; no email/Slack sent implicitly. Same-branch dependent commits wait; unrelated branches/projects continue.

- **SameCellConflictNeedsReview** (`ac-SameCellConflictNeedsReview`): Given base priority=low, A1 priority=high and B1 priority=medium, when B integrates A1, then needs_review shows low/medium/high, B/main remains B1 and no public ref is overwritten.
- **DeleteUpdateNeedsReview** (`ac-DeleteUpdateNeedsReview`): Given A1 deletes T1 and B1 edits T1.title, when histories merge, then needs_review retains B1 and the deleted/edited alternatives for review.
- **CleanMergeCanFailDomainValidation** (`ac-CleanMergeCanFailDomainValidation`): Given separate commits add T1 dependsOn T2 and T2 dependsOn T1, when Dolt merges without cell conflicts, then domain validation records dependency_cycle and leaves the prior valid live head unchanged.
- **StaleReviewRejected** (`ac-StaleReviewRejected`): Given a resolution preview binds local=B1 and incoming=A1, when incoming advances to A2, then resolve returns409 stale_review with no merge commit.
- **ReviewAgeVisible** (`ac-ReviewAgeVisible`): Given unresolved needs_review at12:00, when the clock reaches12:15, then plugins/status and the operator inbox show warning with ReviewId and age900s; at24h severity is critical, with at most one repeat per hour.
- **UnrelatedPublicationContinues** (`ac-UnrelatedPublicationContinues`): Given Atlas/main is blocked by review R1, when Atlas/feature publishes F2 and another project publishes X1, then both succeed while Atlas/main remains unchanged.
- **SafeTodoMergeRules** (`ac-SafeTodoMergeRules`): Given A changes title and B changes description with no domain violation, when merge runs, then both values survive; competing priority or completion edits produce needs_review instead of choosing a timestamp winner.

8. recovery without lost or repeated work

FR-MCP-181 / TR-MCP-FEDERATION-008 / TEST-MCP-229. Phase: Ph5/Ph6. Workflows: U06, U07, U09, U15.

Recover synchronization, receipts and restored hub state without losing local work, inventing success or replaying business effects.

OperationReceipt binds requestId, node, project, branch, hubEpoch, candidate/result hashes and outcome. Durable refs pin candidates/reviews before receipt completion. Worker claims are fenced by WorkerEpoch; reconnect never trusts stored generation across HubEpoch. Fetch may advance remote-tracking refs, never dirty live refs. Restore first disables ingress, restores compatible data/control/audit checkpoints, creates new epoch, then re-advertises verified heads; node-held missing commits are uploaded and ancestry-validated before normal synchronization resumes.

- **LostPushAckDoesNotDuplicateCommit** (`ac-LostPushAckDoesNotDuplicateCommit`): Given remote accepted A1 and reply was lost, when request R1 retries, then it returns A1 and the existing receipt; no A2 commit is created.
- **OldWorkerEpochCannotReportSuccess** (`ac-OldWorkerEpochCannotReportSuccess`): Given worker epoch changes4 to5, when epoch4 reports success, then409 stale_worker_epoch leaves current receipt and desired state unchanged.
- **DirtyBranchDefersApply** (`ac-DirtyBranchDefersApply`): Given B/main=C0 with title=draft, when A1 is fetched, then outcome=waiting_local_work, main stays C0 and title remains draft; at15min status/inbox shows the age warning.
- **IncompatibleSchemaDoesNotApply** (`ac-IncompatibleSchemaDoesNotApply`): Given B supports schema1 and incoming S1 requires schema2, when sync runs, then409 incompatible_schema leaves B's live data/head unchanged and records the required version.

9. collision-resistant and preserved identity

FR-MCP-182 / TR-MCP-FEDERATION-009 / TEST-MCP-230. Phase: Ph2/Ph3/Ph5/Ph6. Workflows: U01, U10, U14.

Prevent generated-ID collisions, preserve permanent legacy aliases, detect duplicate work and atomically reserve human-assigned IDs.

Generated IDs use immutable two-character creator tags and durable branch-independent sequences with per-family grammar fixed in Ph0. Alias key=(ProjectId,OriginNodeId,EntityKind,LegacyId) maps permanently to CanonicalId; ambiguous unqualified lookup returns409 ambiguous_legacy_id. Historical text/commits are not rewritten. Migration duplicate candidates use normalized title/description/dependency features, returning similarity score plus evidence, never auto-merging. Human ID reservations are unique(ProjectId,EntityKind,HumanId) at the hub, bound to node/requestId. Offline nodes use generated provisional IDs; an unreserved human ID is not publishable.

- **TwoNodesCreateDistinctIds** (`ac-TwoNodesCreateDistinctIds`): Given tags LA and LG and both counters start1, when both create a TODO, then generated IDs differ and both records survive reconciliation.
- **BranchRevertDoesNotReuseSequence** (`ac-BranchRevertDoesNotReuseSequence`): Given LA allocated sequence42, when main reverts and another branch creates a record, then its sequence is greater than42.
- **LegacyCollisionPreservesReferences** (`ac-LegacyCollisionPreservesReferences`): Given LAB and LEGION have different entities named TODO-7, when migration resolves the collision, then scoped aliases resolve each original identity and every relational reference follows its reviewed canonical mapping; historical text remains byte-identical.
- **LegacyIdsRemainValid** (`ac-LegacyIdsRemainValid`): Given existing IMPL-UNITGREEN-001 and old TODO references, when imported and read by their original IDs, then they resolve through permanent aliases without rewriting commits, PR text, session logs, memories or documents.
- **DifferentIdsSameTodoDetected** (`ac-DifferentIdsSameTodoDetected`): Given LAB TODO-7 and LEGION TODO-91 share normalized title Fix login, description and dependency set, when migration plan runs, then it emits a duplicate candidate with both IDs and evidence; until explicit review, neither is merged or deleted.
- **SimultaneousHumanIdReservation** (`ac-SimultaneousHumanIdReservation`): Given human ID IMPL-UNITGREEN-001 is unreserved, when two nodes reserve it concurrently, then exactly one receives201 and the other409 human_id_reserved; retry by the winning requestId returns the same reservation.
- **OfflineHumanIdPublicationDenied** (`ac-OfflineHumanIdPublicationDenied`): Given an offline-created provisional TODO requesting an unreserved human ID, when it reconnects and publishes, then409 human_id_unreserved retains the local record until reservation or reviewed alias resolution.

10. first-party plugin federation contract

FR-MCP-183 / TR-MCP-FEDERATION-010 / TEST-MCP-231. Phase: Ph7. Workflows: U01, U02, U05, U07, U08, U11, U12, U13, U14, U15.

Expose consistent typed federation, review, identity, audit, migration and maintenance workflows through every first-party plugin.

All plugins call typed server/client/MCP/REPL contracts; no raw SQL/CLI fallback. Status DTO includes projectId, branch, localHead, publishedHead, observedUtc, dirty, ahead, behind, publicationState, incomingTarget, connectivity, reviewCount, oldestReviewAgeSeconds, waitingLocalWorkAgeSeconds, auditPendingCount, hubEpoch, workerEpoch. Capability contract version1 is negotiated; unknown version=409 unsupported_contract. Administrative maintenance.preflight/attest/status and migration commands share the same typed service as Nuke. Stable error envelope={code, message, requestId, projectId, branch, retryable, details}; status codes are locked in the technical contract appendix.

Technical contract appendix (TR-MCP-FEDERATION-010): all below routes are under /qbrainai/federation unless the commits route specifies the project/branch path. Every mutation requires requestId UUID and authenticated subject; node network requests additionally carry the project-scoped reference token. Responses carry contractVersion=1 and the stable error envelope. Path/body ProjectId mismatch is403 unauthorized_project. No endpoint accepts credentials in URL, argv or persisted receipts.

Read/status routes: GET capabilities returns200 {contractVersion,capabilities}; GET projects/{projectId}/status returns200 the status DTO; GET projects/{projectId}/branches returns200 {items: [{name,head,dirty,publishedHead}]}; GET conflicts and GET conflicts/{reviewId} return200 review records filtered by membership. POST control/poll returns200 {hubEpoch,subscriptions}; reads never acknowledge work. POST control/report validates observed generation and worker epoch, returns200 the durable receipt or409 the named fence error. Poll/report use control scope; reads use read scope.

Mutation routes: POST enroll returns201 the enrollment (or200 the same idempotent enrollment); POST projects/{projectId}/branches creates201 {branch,head}; POST projects/{projectId}/branches/select returns200 {sessionId,branch,head} after local lease validation; POST projects/{projectId}/branches/{branch}/commits returns201 {commitHash,publicationState}, or200 the same commit on retry. POST publication/accept returns200 {requestId,hubEpoch,publishedHead,publishedGeneration,receiptId} only after the guarded ref update is durable. POST branches/merge and POST conflicts/{reviewId}/resolve bind expected heads and return201 the new commit; a conflict returns202 {outcome:needs_review,reviewId} while the live head stays unchanged. Share/merge/publication require publish scope plus branch permission. Enrollment requires enroll scope and the operator owner-binding approval.

POST audit/batches accepts {requestId,projectId,records:[AuditRecord]}, returns200 {receipts: [{originNodeId,recordId,payloadHash,outcome,receivedUtc}]}, and requires audit scope. Every record must match the token/project and batch ProjectId. Any invalid record rejects the whole batch before insert. Exact duplicate is already_received; payload mismatch is409 audit_id_conflict. POST ids/reservations accepts {requestId,projectId,entityKind,humanId,nodeId}, requires publish scope and returns201 the reservation or200 its exact retry; another owner gets409 human_id_reserved. POST maintenance/preflight and POST maintenance/attest, GET maintenance/status and POST migration/{plan|dry-run|run|resume|verify|export-todos|cutover|abort}, GET migration/status are operator-only local service operations. They are exposed through typed clients and Nuke, never public node-admin capabilities. A long migration returns202 {migrationId,status}; status/verify return200 a receipt, and a denied readiness check never performs cutover.

Stable failure mapping:401 invalid_token;403 unauthorized_project;409 head_changed, working_set_changed, lease_lost, branch_busy, stale_review, stale_worker_epoch, hub_epoch_changed, ambiguous_legacy_id, human_id_reserved, human_id_unreserved, target_not_empty, maintenance_not_ready, migration_source_changed, migration_verification_failed, incompatible_schema, unsupported_contract and

publication_required;422 secret_detected, dependency_cycle and unsupported_provider;501 unsupported_capability;503 identity_unavailable or remote_unavailable. TLS rejection produces the local transport code tls_peer_untrusted with no HTTP response and no request sent. Missing required fields return400 invalid_request. Missing authorized resources return404 not_found; project authorization precedes existence disclosure. Malformed/unsupported token is401; valid token lacking project rights is403.

Outcome fields are not HTTP errors: committed_locally, already_received, waiting_local_work and needs_review remain visible through200 status/report responses. A requested live mutation blocked by drafts returns409 waiting_local_work; an integration that durably creates review returns202 needs_review. dependency_cycle is a validationErrors entry on that review as well as422 for a directly invalid local mutation. retryable=true is restricted to transient503/transport failures; all stale-head/fence/review errors require reread and a fresh validated action. Idempotent retries with the same requestId but different payload return409 request_id_conflict. No automatic retry creates a different commit.

Test fixtures capture actual content hashes as symbols C0,A1,B1,H1,M1,M2,S1 rather than hardcoding unstable engine hashes. Each symbol is immutable within the run manifest. Given/When/Then compares exact hash equality, specified parent sets, row values, status codes and error codes against that manifest; a successful result must never be inferred solely from an empty queue. Capture UTC timestamps and a fake clock for exact15-minute/24-hour age boundaries. All plan phase IDs use Ph0-Ph8; priority values remain separate.

- **EveryPluginUsesSameContract** (`ac-EveryPluginUsesSameContract`): Given one fixture for commit R1, conflict R2 and migration R3, when each installed first-party plugin calls its wrapper, then normalized IDs, heads, error codes and audit agent identity match the typed client result.
- **OldServerReportsUnsupportedCapability** (`ac-OldServerReportsUnsupportedCapability`): Given server lacks federation contract1, when a plugin invokes commit, then unsupported_contract is shown and no raw SQL or shell fallback executes.
- **ReconnectRevalidatesContext** (`ac-ReconnectRevalidatesContext`): Given a cached leaseFence4 and session branch feature, when reconnect discovers fence5, then the plugin retains visible context but refuses writes with lease_lost until reacquired.
- **StaleBranchReportedNotDeleted** (`ac-StaleBranchReportedNotDeleted`): Given a branch observation is older than24h, when status is displayed, then stale=true and observedUtc are visible; the branch, drafts and refs are not deleted.

11. portable data versus host-local state

FR-MCP-184 / TR-MCP-FEDERATION-011 / TEST-MCP-232. Phase: Ph2/Ph3/Ph5/Ph7. Workflows: U10, U13.

Separate versioned project content from operational state while delivering permanent audit to the hub independently of commits and excluding secrets.

Project history excludes secrets and automatic operational audit. This does NOT make audit local-only: every host retains permanent local audit and an append-only outbox delivers sanitized records to the hub unversioned ledger independently of project commits. AuditRecord key=(OriginNodeId,RecordId UUID); fields ProjectId,EntityKind,EntityId,OriginSequence int64,OccurredUtc,Actor,Action,PayloadVersion,PayloadHash,Payload. POST audit/batches validates token/project, inserts atomically by key and returns per-record durable receipts. Same key/hash is already_received; same key/different hash=409 audit_id_conflict. No UPDATE/DELETE permission. Local pruning is forbidden. Migration backfills TODO and generic audit history with counts/hashes and receipts. Credential values are excluded at capture, not silently replicated.

- **CloneOmitsSecrets** (`ac-CloneOmitsSecrets`): Given credentials and local machine paths exist in operational storage, when a project is cloned, then none appears in its versioned tables or history.

- **AuditReachesHub** (`ac-AuditReachesHub`): Given A performs an offline TODO edit and creates audit E1, when connectivity returns without a Dolt commit, then hub audit contains exactly E1, A retains E1, and main/remote remain C0.
- **AuditRetryIdempotent** (`ac-AuditRetryIdempotent`): Given E1 was stored but its receipt was lost, when the same batch retries, then exactly one E1 exists and the same hash receipt returns; changed payload under E1 returns409 audit_id_conflict.
- **AuditBackfillVerified** (`ac-AuditBackfillVerified`): Given source has100 TODO audit and20 generic audit rows, when seed/join audit backfill verifies, then all120 records have matching payload hashes and hub receipts; missing one blocks cutover.
- **FullProjectSessionGraphPreserved** (`ac-FullProjectSessionGraphPreserved`): Given an explicitly promoted sanitized session with turns, actions, dialog, commits/files, PlanFile and TodoId, when cloned, then every promoted child and relationship matches the source snapshot; automatic capture still uses the audit channel.
- **AcceptedHeadRebuildsBranchIndex** (`ac-AcceptedHeadRebuildsBranchIndex`): Given accepted head changes C0 to A1, when the derived index rebuild completes, then queries read A1 values and no C0 cache is served under the A1 key.
- **BackgroundWritersDoNotDirtyVersionedBranch** (`ac-BackgroundWritersDoNotDirtyVersionedBranch`): Given main=C0 is clean, when heartbeat, audit and automatic session capture run, then Dolt working-set fingerprint remains unchanged.
- **SecondAgentLoggingNeedsNoLease** (`ac-SecondAgentLoggingNeedsNoLease`): Given agent1 owns the editing lease, when agent2 logs a session action, then its local audit persists without acquiring or changing agent1's lease.
- **CommitPreviewBlocksSecret** (`ac-CommitPreviewBlocksSecret`): Given a promoted snapshot contains a fixture secret detected by the configured detector, when commit preview runs, then422 secret_detected creates no commit and outputs no secret value.
- **ReceiptsAndLogsContainNoCredentials** (`ac-ReceiptsAndLogsContainNoCredentials`): Given unique canary values are injected through protected secret references, when commands, retries and failures execute on both hosts, then scanning argv, receipts, logs, chat/audit payloads and generated files finds zero canary values.

12. scoped authorization and offline behavior

FR-MCP-185 / TR-MCP-FEDERATION-012 / TEST-MCP-233. Phase: Ph0/Ph2/Ph5. Workflows: U01, U06, U13.

Reuse integrated IdentityServer for single-user node ownership, per-node client credentials, project scopes and revocation; preserve authorized offline work.

Reuse existing Duende IdentityServer. Provision client fed.node.{NodeId} per single-user node, bound to ownerSubjectId through authenticated operator enrollment. Client credentials issue short-lived reference access tokens; audience qbrain-federation and scopes fed.project.{ProjectId}.{read|publish|control|audit|enroll}; protected actions additionally require server membership/branch permission. Gateway introspects without positive caching for each new request and rechecks before publication commit; unavailable identity returns503 identity_unavailable. Revoke through IdentityServer: disable client and remove its persisted reference grants. Gateway rejects inactive tokens401 invalid_token; do not invent a revocation ledger. Native remotesapi uses gateway-held credentials over loopback; clients cannot access it directly. Ph0 proves token injection/relay including redirects/object URLs, project isolation and no public-ref bypass.

- **RawRemoteCannotBypassPublicationPolicy** (`ac-RawRemoteCannotBypassPublicationPolicy`): Given valid A read credentials, when A attempts direct backend access or public-main push, then the backend is unreachable across the link or403 publication_required; remote/main remains C0.
- **ForeignProjectCloneDenied** (`ac-ForeignProjectCloneDenied`): Given token has Atlas.read but no SecretProject scope, when a remote fetch requests SecretProject, then403 unauthorized_project sends zero objects.
- **RevokedNodeCannotPublish** (`ac-RevokedNodeCannotPublish`): Given A holds a reference token, when IdentityServer disables A and revokes its grants before acceptance, then accept returns401 invalid_token and remote/main is unchanged.
- **OfflineLocalAuthorizationStillApplies** (`ac-OfflineLocalAuthorizationStillApplies`): Given hub/IdentityServer is unreachable and a valid authorized local user, when a local TODO edit is requested, then it succeeds; an unauthorized local user still receives403.
- **LocalAppUserCannotPush** (`ac-LocalAppUserCannotPush`): Given ordinary editing SQL credentials, when remote administrative procedures or public-ref writes are attempted, then permission is denied and no object publication occurs.
- **RevokedCredentialFailsFetchAndPublish** (`ac-RevokedCredentialFailsFetchAndPublish`): Given IdentityServer revocation completes, when A starts new fetch and publish requests with its old token, then both return401 invalid_token before transfer or ref mutation.
- **SqlPortUnreachableAcrossLink** (`ac-SqlPortUnreachableAcrossLink`): Given both services run, when LAB probes10.42.0.143:3306 and LEGION probes10.42.0.214:3306, then both connections fail; local127.0.0.1:3306 health succeeds.
- **UnknownCertificatePinRejected** (`ac-UnknownCertificatePinRejected`): Given the peer uses an unapproved SPKI or wrong SAN, when control/remote TLS connects, then tls_peer_untrusted occurs before bearer transmission.

13. Dolt fourth-provider support and schema migration

FR-MCP-186 / TR-MCP-FEDERATION-013 / TEST-MCP-234. Phase: Ph0/Ph1/Ph6/Ph8. Workflows: U11.

Support Dolt as the fourth provider with dedicated schema migration, controlled schema-history distribution, Windows/Linux lifecycle and original-provider regressions.

Provider key dolt has a dedicated migration assembly; original provider migrations remain unchanged. Versioned project schema migrations are authored only by hub maintenance on maintenance/schema-N and distributed as history. MaintenanceReadiness binds epoch, binary/schema versions, all local branches including unpublished ones, heads, working-set hashes and attestations. Preflight must pass before Nuke swaps binaries; dirty/unattested branch=409 maintenance_not_ready. LAB uses pinned Linux image; LEGION uses checksum-pinned native Windows dolt.exe supervised by its existing Windows service, local paths on F: and exclusive Windows file handle/named mutex ownership. All B1-B12 are evidenced on both OSs.

- **FourProvidersResolve** (`ac-FourProvidersResolve`): Given keys sqlite,postgresql,sqlserver,dolt, when runtime and design-time factories resolve them, then each selects its own strategy/migration history; unknown key returns unsupported_provider.
- **DoltBaselineAndUpgrade** (`ac-DoltBaselineAndUpgrade`): Given empty standalone Dolt then baseline schema1, when baseline and additive schema2 apply, then CRUD succeeds, data remains intact and repeated apply makes no further changes.
- **OriginalProviderMigrationsUnchanged** (`ac-OriginalProviderMigrationsUnchanged`): Given recorded hashes of the three original migration chains, when Dolt support is added, then those hashes match and all three real-provider upgrade fixtures pass.

- **UnsupportedSchemaRejected** (`ac-UnsupportedSchemaRejected`): Given application supports schema1, when opening schema2 for domain writes, then incompatible_schema is returned before mutation.
- **VersionedProjectNotMigratedAtStartup** (`ac-VersionedProjectNotMigratedAtStartup`): Given a versioned project at schema1, when application starts, then no EF migration executes and its Dolt head/schema remain unchanged.
- **OldSchemaUnpublishedCommitMergesAfterUpgrade** (`ac-OldSchemaUnpublishedCommitMergesAfterUpgrade`): Given B1 is unpublished on schema1 and hub S1 adds schema2 from C0, when preflight/upgrade and merge succeed, then B/main=M1 with both B1 and S1 ancestors and B1 row values preserved.
- **FeatureBranchSchemaCatchUp** (`ac-FeatureBranchSchemaCatchUp`): Given main accepts S1 but feature still uses schema1, when feature is selected for writes, then it remains closed until its own merge with S1 validates.
- **DirtyDraftBlocksBinaryUpgrade** (`ac-DirtyDraftBlocksBinaryUpgrade`): Given any local branch has a draft, when Nuke preflight runs, then409 maintenance_not_ready prevents binary replacement and preserves the draft.
- **ContractBlockedUntilLocalBranchesAttested** (`ac-ContractBlockedUntilLocalBranchesAttested`): Given unpublished feature lacks an expand-schema attestation, when contract migration is requested, then it is refused even if all public remote branches are current.
- **DeploymentPreflightBlocksDirtyOrUnattestedBranch** (`ac-DeploymentPreflightBlocksDirtyOrUnattestedBranch`): Given a ready public main but an unattested local feature, when deployment runs, then no binary/service/schema change occurs; explicit branch attestation is required.
- **WindowsWriterLockAndRestart** (`ac-WindowsWriterLockAndRestart`): Given one Windows worker owns Atlas, when a second process starts or the owner crashes, then concurrent ownership is denied and restart recovers only after the OS releases its lock; committed rows and unpublished heads survive.

14. original-provider-to-Dolt migration tool

FR-MCP-187 / TR-MCP-FEDERATION-014 / TEST-MCP-235. Phase: Ph3 seed/export; Ph6 join. Workflows: U10, U14.

Migrate all three original providers using verified seed then reviewed join, permanent identities, audit backfill and a source-retaining TODO-to-SQLite recovery export.

MigrationManifest binds MigrationId, mode, ProjectId, source provider/snapshot hash, schema versions, table counts, typed row hashes, alias decisions, audit backfill and immutable destination commit. Ph3 implements seed, dry-run/resume/verify and Dolt-to-SQLite TODO export; join is unavailable until Ph6 review/publication exist. Lab sources are both SQLite; PostgreSQL and SQL Server are supported fixture sources. Retain source SQLite read-only. Export freezes a chosen Dolt head and writes a NEW SQLite recovery DB containing TODO items/tasks/lists/links/document metadata/aliases and audit linkage, with compatible schema and verification receipt; never overwrite retained source. Export is not full-database reverse migration. Cutover uses a verified receipt, explicit DEPLOY HOLD release and one-node soak before the second host.

- **ThreeSourceProviderRoundTrip** (`ac-ThreeSourceProviderRoundTrip`): Given the same fixture in SQLite,PostgreSQL,SQL Server, when seed migration runs, then counts, typed hashes, IDs, nulls, Unicode, decimals, UTC precision and FK relationships match in Dolt.
- **ResumeAfterBatchCommit** (`ac-ResumeAfterBatchCommit`): Given target batch1 committed before a crash but its acknowledgement was lost, when resume runs, then batch1 is recognized by journal and no duplicate rows appear.

- **ChangedSnapshotRejected** (`ac-ChangedSnapshotRejected`): Given manifest source hash S0, when source is replaced by S1 before resume, then409 migration_source_changed writes no additional target rows.
- **FailedVerifyCannotCutover** (`ac-FailedVerifyCannotCutover`): Given one row hash differs, when cutover is requested, then409 migration_verification_failed leaves source routing active and writes closed on the target.
- **SourceRetainedAfterCutover** (`ac-SourceRetainedAfterCutover`): Given verified cutover completes, when target receives a new TODO, then source backup remains present, read-only and byte-identical; the new TODO exists only in Dolt until an explicit recovery export.
- **MigrationJoinPreservesCommonAncestor** (`ac-MigrationJoinPreservesCommonAncestor`): Given seed C0 is published and join imports J1, when join completes in Ph6, then C0 is an ancestor of J1 and reviewed main M1; no second root is created.
- **MigrationJoinDivergentRowRequiresReview** (`ac-MigrationJoinDivergentRowRequiresReview`): Given seed T1.title=A and frozen join T1.title=B, when join planning runs, then an explicit keep/take/manual decision is required before J1 even if Dolt would fast-forward.
- **JoinCutoverRequiresReviewedMain** (`ac-JoinCutoverRequiresReviewedMain`): Given J1 is verified but main integration is unresolved, when cutover runs, then409 migration_verification_failed preserves source routing; only verified reviewed M1 permits cutover.
- **DoltTodoExportPreservesNewWrites** (`ac-DoltTodoExportPreservesNewWrites`): Given A1 adds TODO T2 after cutover, when export-sqlite targets A1, then the new SQLite DB contains T2 and all TODO children/aliases with matching hashes, while the original source remains unchanged.
- **SeedPhaseRejectsJoin** (`ac-SeedPhaseRejectsJoin`): Given only Ph3 is implemented, when mode=join is requested, then409 unsupported_capability creates no target commit or routing change.

15. imported state does not repeat external effects

FR-MCP-188 / TR-MCP-FEDERATION-015 / TEST-MCP-236. Phase: Ph2/Ph6. Workflows: U04, U09, U13.

Imported project or audit state never replays external effects and general federation remains outside QuadBrain-only keyserver gating.

History import and audit ingestion apply validated state, never dispatch original domain commands or external effects. EffectIdentity is stable across retries/import. Imported operational metadata is inert evidence, not an execution request. General federation apply/control is outside QuadBrain keyserver transaction gating per FR-MCP-173; actual QuadBrain operations retain their separate coordinator contract. Legacy HMAC envelopes remain isolated compatibility behavior.

- **ImportDoesNotSendExternalAction** (`ac-ImportDoesNotSendExternalAction`): Given A1 records an already-sent GitHub update, when B integrates A1, then B state includes the result and outbound GitHub/email invocation count is0.
- **RetryDoesNotRepeatEffect** (`ac-RetryDoesNotRepeatEffect`): Given accepted commit A1 and effect E1, when sync retries10 times, then the effect ledger still has one E1 and no new external dispatch occurs.
- **ImportedOperationalRowsRemainInert** (`ac-ImportedOperationalRowsRemainInert`): Given audit backfill contains command names and receipt payloads, when the hub ingests them, then they are stored as evidence and no business command executes.
- **FederationBypassesQuadBrainKeyserver** (`ac-FederationBypassesQuadBrainKeyserver`): Given QuadBrain coordinator is degraded but local authorization passes, when federation commit/control/audit operations execute, then none invokes the keyserver and valid local work succeeds.

16. safe shared-host editing and commit ownership

FR-MCP-189 / TR-MCP-FEDERATION-016 / TEST-MCP-237. Phase: Ph2/Ph4/Ph6. Workflows: U07, U12.

Enforce branch edit ownership and fenced live-write serialization so no agent or sync worker captures or overwrites another agent draft.

BranchLease key=(ProjectId,Branch), OwnerSessionId, Fence int64, ExpiresUtc and WorkingSetFingerprint. Every live-branch mutation holds the same host-owner write gate across lease validation and transaction/commit. Expiry rejects new admission; an admitted write finishes before takeover. Isolated worker rights cover only non-live integration refs and it invokes owner API for live moves. Dirty takeover needs explicit review. Commit requestId persists in history for idempotency; preview binds exact head/fingerprint.

- **CommitCannotCaptureOtherAgentDraft** (`ac-CommitCannotCaptureOtherAgentDraft`): Given agent1 owns a draft, when agent2 commits the same branch, then409 branch_busy creates no commit and the draft remains agent1-owned.
- **ExpiredLeaseFenced** (`ac-ExpiredLeaseFenced`): Given fence4 expires and fence5 is issued, when a fence4 edit arrives, then409 lease_lost persists no row changes.
- **PreviewChangeRequiresReview** (`ac-PreviewChangeRequiresReview`): Given preview fingerprint W1, when the working set becomes W2, then commit with W1 returns409 working_set_changed and requires a fresh preview.
- **LeaseLostDuringWriteRejected** (`ac-LeaseLostDuringWriteRejected`): Given a write admitted under fence4, when expiry occurs mid-transaction, then that write finishes under the gate before fence5 takeover; a later fence4 write is rejected.
- **CommitRetryReturnsSameHash** (`ac-CommitRetryReturnsSameHash`): Given request R1 created A1 and reply was lost, when R1 retries after restart, then A1 is returned by its commit trailer and history contains no duplicate commit.
- **IncomingMergeCannotRaceAdmittedEdit** (`ac-IncomingMergeCannotRaceAdmittedEdit`): Given an edit holds the branch gate, when the integration worker requests a live head move, then it waits; after the edit finishes, fingerprint revalidation detects the draft and returns waiting_local_work without overwriting it.

Applicable compatibility and migration acceptance

The following current criteria close requirements recovered in the gap check. They are limited to applicable behavior, source-data preservation and explicit existing-mode regressions. They do not reinstate business-command forwarding or read switching for Dolt projects. A criterion shared by records is printed once with all source IDs.

- **TEST-MCP-FED-ROUTE-001/review20261110-1**: Given A Atlas and B SecretProject, when A supplies B path or hash, then403 unauthorized_project sends no B rows and performs no mutation.
- **FR-MCP-103/review20261110-1**: Given A cannot reach the hub, when it edits and commits T1, then local read returns the change and publication remains pending.
- **FR-MCP-FED-WSSYNC-001/review20261110-1**: Given A changes host RunAs or DataDirectory, when heartbeat/clone runs, then those values never replace H configuration.
- **FR-MCP-MEMORY-008/review20261110-2**: Given A deletes and B edits M1, when merged, then needs_review preserves both histories.
- **TR-MCP-FED-ROUTE-001/review20261110-1**: Given a federation hop with unresolved workspace in legacy mode, when routed, then a4xx diagnostic is returned and no primary workspace fallback occurs.

- **FR-MCP-MEMORY-008/review20261110-3**: Given a global memory containing machine configuration, when cloning a project, then that memory is absent.
- **FR-MCP-FED-ROUTE-001/review20261110-1**: Given A is enrolled in Atlas, when its control request omits ProjectId or requests another project, then400 project_required or403 unauthorized_project returns no default-workspace data.
- **TEST-MCP-136/review20261110-1**: Given a local commit during outage, when reconnect runs, then history is exchanged without replaying the original command.
- **FR-MCP-FED-BACKFILL-001/review20261110-3**: Given a missing source row or audit receipt, when verify runs, then migration_verification_failed blocks cutover.
- **TR-MCP-AUDIT-004/review20261110-3**: Given a wiki export or project clone, when generated, then it excludes credentials and operational audit rows while independent hub delivery remains enabled.
- **FR-MCP-FED-WSSYNC-001/review20261110-3**: Given an authorized portable prompt-template edit is committed, when synced, then peers receive its sanitized value.
- **TR-MCP-AUDIT-004/review20261110-1**: Given batchE1 was accepted and its reply lost, when retried, then one row and the same payload-hash receipt remain.
- **FR-MCP-FED-BACKFILL-001/review20261110-2**: Given completed batches, when rerun, then no duplicate rows or external effects occur and already_applied counts are reported.
- **FR-MCP-FED-WSSYNC-001/review20261110-2**: Given CurrentRequirementLayerKey differs, when heartbeat runs, then hub-owned value remains and drift is reported.
- **FR-MCP-125/review20261110-2**: Given degraded QuadBrain coordinator, when authorized general federation control or audit runs, then no keyserver call occurs and the operation is not blocked by coordinator state.
- **TEST-MCP-MEMORY-FED-001/review20261110-1**: Given different same-base edits to M1.text, when sync merges, then needs_review returns base/local/incoming values and retains the valid local head.
- **TR-MCP-AUDIT-004/review20261110-2**: Given E1 key with a changed payload, when submitted, then409 audit_id_conflict performs no overwrite.
- **TR-MCP-FED-001/review20261110-2**: Given explicit DirectProxy mode, when a foreign workspace is addressed, then correct routing and loop protection remain intact.
- **TR-MCP-TXNCOMPAT-001/review20261110-1**: Given federation apply/control operations and RequiredForMutations=true, when executed, then ITurnTransactionCoordinator invocation count is0; brain-slot.invoke remains gated.
- **FR-MCP-103/review20261110-2**: Given H and A commit concurrently, when synchronization completes, then both histories survive or a visible review blocks only that branch.
- **TEST-MCP-FED-WSSYNC-001/review20261110-1**: Given H layer release and A layer draft, when A heartbeats, then H remains release and drift=true; only a separately authorized policy commit changes it.
- **FR-MCP-AUDIT-001/review20261110-3**: Given legacy and new compressed payload fixtures, when read/migrated on all supported providers, then their decoded values and historical text remain unchanged.
- **FR-MCP-FED-ROUTE-001/review20261110-2**: Given legacy proxy routing is explicitly selected, when valid normalized path or GlobalWorkspaceId is supplied, then both resolve the same scoped workspace.
- **FR-MCP-125/review20261110-1**: Given legacy signed envelopes, when explicit legacy-mode tests run, then compatibility remains intact.
- **TR-MCP-FED-MEMORY-001/review20261110-1**: Given MemoryId M1 with versions V1,V2 in Atlas, when A1 is cloned, then both versions and their links match and no global memory is included.

- **TR-MCP-FED-BACKFILL-001/review20261110-1**: Given migration J1 contains all source rows and audit records, when verify executes, then source accounting at J1 and final invariants at M1 both pass, with durable audit receipts.
- **FR-MCP-AUDIT-001/review20261110-1**: Given offline audit E1, when reconnect occurs without a version commit, then the hub stores E1 once and the node retains it.
- **TR-MCP-FED-WSSYNC-001/review20261110-1**: Given older legacy registration lacks fields, when processed, then its existing mode defaults remain without mutating versioned project settings.
- **FR-MCP-AUDIT-001/review20261110-2**: Given runtime storage credentials, when update/delete of any existing audit row is attempted, then it is denied.
- **TEST-MCP-AUDIT-003/review20261110-1**: Given source has100 TODO audit and20 generic audit records, when backfill verifies, then all120 matching hashes and receipts exist at the hub; missing one blocks cutover.
- **FR-MCP-FED-BACKFILL-001/review20261110-1**: Given the domain manifest, when migration planning runs, then every source table/writer has a classified destination or explicit exclusion and no unknown domain passes.
- **TR-MCP-FED-001/review20261110-1**: Given versioned mode, when a TODO request arrives, then LocalProxy middleware is not invoked.
- **FR-MCP-MEMORY-008/review20261110-1**: Given workspace memory M1 edited offline, when saved, then the local read changes but remote C0 does not; after explicit commit and sync the peer receives the version.
- **TEST-MCP-FED-BACKFILL-001/review20261110-1**: Given100 source TODO rows and120 audit records, when migration repeats after acknowledgement loss, then destination counts remain100/120, original timestamps/IDs resolve and external dispatch count is0.

Required fixture coverage

Ph3 migration fixtures cover every classified project domain and its child/link graph, soft-deleted rows, multiple projects, decimal/UTC/binary/Unicode edge cases, promoted session snapshots, audit records, derived-index rebuild and inert operational metadata. ResumeAfterBatchCommit injects death after target-batch commit before acknowledgement. FailedVerifyCannotCutover additionally tests death before provider mapping switch and after switch before reopening: recovery opens exactly one verified provider, retains source bytes and never loses Dolt-only writes. Dry-run and failed verification cannot alter source or routing. Ph6 repeats these invariants for reviewed joins.

Path fixtures retain syntax-aware Windows/UNC/Linux normalization and containment, no host-native reinterpretation of a foreign path, and backed-up transactional source rekey where required. Session fixtures preserve every field/child, PlanFile/TodoId, unknown/missing-session behavior and source sanitization parity. Memory fixtures preserve IDs, scope, ownership, versions and soft deletion. Configuration fixtures retain hub-admin requirement-layer policy, report drift, and keep RunAs/paths/tunnels host-local. These are applicable migration/security constraints, not additional historical feature work.

Federation current-scope coverage

This index covers the new Dolt federation release. The architecture, phase plan and U01-U15 workflows define scope. Earlier plans were consulted to recover applicable requirements; their original text and unrelated requirements are not part of this plan.

Release contracts and acceptance criteria

FR-MCP-174 through 189 map in order to TR-MCP-FEDERATION-001 through 016 and TEST-MCP-222 through 237. The workflow inventory names each contract, its phase, workflows and acceptance tests. Together these define 91 current Given/When/Then cases. The current contract and acceptance specification includes the full schemas, endpoint/error contracts and Given/When/Then criteria once in this packet, with canonical FR/TR/TEST IDs. Applicable gap-check criteria and the migration fixture matrix are included there too. All criteria remain pending implementation evidence.

U01-U15 cover enrollment, local/hub commits, concurrent work, conflicts, disconnect/reconnect, drafts, branches, retries, migration, schema upgrades, multiple agents, audit delivery, duplicate identity and hub restore. Their preconditions, steps, success outcomes and exception paths are in the workflow document. Ph0 registers these workflows in the UC store and binds their requirement links; no pre-existing federation UC was found in either queried workspace.

Applicable requirements recovered by the gap check

- **Autonomous execution and compatibility:** FR-MCP-103, TR-MCP-FED-001 and TEST-MCP-136 now require local execution with committed-history exchange. FR-MCP-077 preserves existing explicitly selected DirectProxy behavior as a regression boundary. Ph2/Ph5; FR-MCP-174/178/185.
- **Permanent audit reaches the hub:** FR-MCP-AUDIT-001, TR-MCP-AUDIT-002/003/004 and TEST-MCP-AUDIT-001/002/003 cover append-only storage, protected runtime permissions, secret exclusion, delivery receipts and migration backfill. Ph2/Ph3/Ph5; FR-MCP-184/187.
- **Existing identity platform:** MCP-FEDIDENTITY-001 supplies user management, single-user node ownership and integrated IdentityServer authentication. Per-node credentials, project scopes and revocation are in FR-MCP-185. Ph2/Ph5/Ph7.
- **Complete migration:** FR-MCP-FED-BACKFILL-001, TR-MCP-FED-BACKFILL-001 and TEST-MCP-FED-BACKFILL-001 require domain inventory, child/link completeness, counts, hashes, idempotent resume and audit verification. Ph3 seed/recovery export and Ph6 reviewed join; FR-MCP-187.
- **Project routing and path identity:** FR-MCP-FED-ROUTE-001, TR-MCP-FED-ROUTE-001, TEST-MCP-FED-ROUTE-001, TR-MCP-FED-PATH-001, TEST-MCP-FED-PATH-001, TR-MCP-PATHNORM-001 and TEST-MCP-PATHNORM-001 retain normalized local paths, project isolation, foreign-path rejection and explicit backed-up source rekey where required. Ph0/Ph2/Ph3/Ph5; FR-MCP-176/182/185.
- **Configuration boundaries:** FR-MCP-FED-WSSYNC-001, TR-MCP-FED-WSSYNC-001 and TEST-MCP-FED-WSSYNC-001 preserve portable configuration, local host settings and protected hub-owned requirement-layer policy with drift reporting. Ph2/Ph5; FR-MCP-184.
- **Memory preservation:** FR-MCP-MEMORY-008, TR-MCP-FED-MEMORY-001 and TEST-MCP-MEMORY-FED-001 preserve memory IDs, versions, ownership, scope and deletion semantics in versioned project history. Ph2/Ph3/Ph6; FR-MCP-184/187.

- **Session graph and sanitization:** TR-MCP-FED-SESSIONLOG-001, TEST-MCP-SESSIONLOG-FED-001 and the applicable SESSIONLOGCTX/SESSIONLOGSAN contracts preserve complete captured graphs, PlanFile/TodoId and sanitizer parity. Automatic audit uses unversioned delivery; explicit promoted snapshots may be versioned. Ph2/Ph3/Ph5; FR-MCP-184/187.
- **No external-effect replay:** FR-MCP-125, FR-MCP-173 and TR-MCP-TXNCOMPAT-001 preserve separation of general federation from QuadBrain-only transaction gating. Import never replays business effects. Ph2/Ph6; FR-MCP-188.
- **Execution gates:** current and prior unit coverage, MigrationIntegrationTests, Build.Tests, provider integration and ValidateTraceability run through Nuke on both hosts at phase exit, with zero failures and zero skips. Ph0-Ph8.

Scope boundary

The new design replaces hub-authoritative ordinary reads/writes, LocalProxy business-command forwarding/replay, post-backfill read switching and general-federation keyserver gating. Those mechanisms are not work items for this release. Required compatibility and source-data migration are limited to the explicit contracts above.

Pull requests, automatic hub election, arbitrary file/source-code synchronization, live cross-provider CDC, unrelated worktree cleanup and unrelated historical requirements are outside this release. Old examples, review transcripts, host-by-host copies and superseded AC wording remain audit evidence only; they do not expand scope or create phase-exit obligations. The canonical current criteria and this plan govern acceptance.